International Centre for Education and Research (ICER)
VIT-Bangalore

Enhancing Stock Price Prediction for Nifty 50 IT Sector Companies: A Comparative Study of ML Models with Event Data Integration

CS6510 – PROJECT 1

REPORT

Submitted by

Aniket Shinde – 24MSP3038

In partial fulfilment for the award of the degree of

POST GRADUATE PROGRAMME

INTERNATIONAL CENTRE FOR HIGHER EDUCATION AND RESEARCH

VIT BANGALORE

DECEMBER, 2023

International Centre for Education and Research (ICER)
VIT-Bangalore

BONAFIDE CERTIFICATE

Certified that this project report **"Enhancing Stock Price Prediction for Nifty 50 IT Sector Companies: A Comparative Study of ML Models with Event Data Integration"** is the bonafide record of work done by **"Aniket Shinde – 24MSP3038"** who carried out the project work under my supervision.

Signature of the Supervisor	**Signature of Director**
Dr. Pitchumani Angayarkanni S	**Prof. Prema M**
Professor,	**Director,**
ICER	ICER
VIT Bangalore	VIT Bangalore.

Evaluation Date:

International Centre for Education and Research (ICER)
VIT-Bangalore

ACKNOWLEDGEMENT

I express my sincere gratitude to our director of ICER **Prof. Prema M.** for their support and for providing the required facilities for carrying out this study.

I wish to thank my faculty supervisor(s), **Dr. Pitchumani Angayarkanni S, Professor**, ICER for extending help and encouragement throughout the project. Without his/her continuous guidance and persistent help, this project would not have been a success for me.

I am grateful to all the members of ICER, my beloved parents, and friends for extending the support, who helped us to overcome obstacles in the study.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	5
	LIST OF FIGURES	6
1.	INTRODUCTION	7
2.	LITERATURE REVIEW	8
3.	OBJECTIVE	9
4.	PROPOSED METHODOLOGY	10
5.	TOOLS AND TECHNIQUES	14
6.	IMPLEMENTATION	16
7.	RESULTS AND DISCUSSIONS	23
8.	CONCLUSION	27
9.	FUTURE ENHANCEMENT	28
10.	APPENDICES	29
11.	REFERENCES	64
12.	WORKLOG	65

ABSTRACT

This study addresses the challenge of predicting stock prices in the volatile IT sector of the Nifty 50 index, where traditional models based solely on historical data often fail to account for influential market events, such as economic policy shifts, earnings reports, and global economic changes. By incorporating event-driven data into predictive models, this research aims to improve the accuracy of stock price forecasting for IT companies, providing a more responsive tool for market analysis. Specifically, we evaluate the performance of ARIMA, SARIMA, GRU, and LSTM models, integrating event-driven and sentiment-based features through feature selection to identify the most impactful variables. The comparative analysis of these models across multiple performance metrics highlights the potential of event-enhanced LSTM models, which demonstrate lower error rates and improved accuracy over baseline approaches. The findings contribute to existing financial literature by demonstrating how the integration of external events and feature optimization can significantly improve stock forecasting accuracy. This study holds practical value for investors, analysts, and financial institutions, supporting more robust decision-making in high-volatility markets and setting a foundation for applying this predictive approach across other sectors.

LIST OF FIGURES

Figure No.	Figure Name	Pg. No.
Fig. 4.1	Methodology	10
Fig. 6.1	Time Series Plot	17
Fig. 6.2	Daily Returns Percentage Change Histogram	18
Fig. 6.3	Correlation Matrix	19
Fig. 6.4	30-Day Rolling Volatility Plot	20
Fig. 7.1	Table comparing evaluation metrics of models	23
Fig. 7.2	Last 10 days vs Predictions for Net 10 days (ARIMA)	24
Fig. 7.3	Last 10 days vs Predictions for Net 10 days (SARIMA)	24
Fig. 7.4	Last 10 days vs Predictions for Net 10 days (GRU)	25
Fig. 7.5	Last 10 days vs Predictions for Net 10 days (LSTM)	25

1. INTRODUCTION

The stock market serves as a critical indicator of global economic health, reflecting the performance of industries and the broader financial landscape. Predicting stock prices has long been an area of interest for investors and analysts, as accurate forecasts enable better decision-making, risk management, and investment strategies. However, the stock market's highly volatile and multifaceted nature makes accurate prediction a significant challenge. External factors such as macroeconomic policies, geopolitical events, and market sentiment intertwine with historical trends, creating a dynamic environment that is difficult to model using traditional methods. This study explores the potential of machine learning to address these challenges and provide a more robust framework for forecasting stock prices.

Historically, stock market analysis has relied on statistical techniques that assume linear relationships in the data. While these methods offer valuable insights, they often fail to capture the intricate, non-linear dependencies that govern financial systems. Advances in machine learning and artificial intelligence have introduced new tools capable of modelling these complexities. Recurrent Neural Networks (RNNs) and their variants, such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), are particularly well-suited to analyzing time-series data like stock prices. By leveraging historical stock data and macroeconomic indicators, these models can uncover patterns that traditional approaches overlook. This research aims to evaluate the predictive capabilities of machine learning models, focusing on their ability to forecast stock prices more accurately than conventional methods.

Central to this study is the question of whether machine learning models can outperform classical techniques in predicting stock prices. To address this, the research evaluates model performance using quantitative metrics such as R-squared and Mean Squared Error (MSE). The models are trained on historical stock market data, including features like opening and closing prices, daily highs and lows, trading volumes, and macroeconomic factors such as gold prices, exchange rates, and indices like the S&P 500. By analyzing these metrics, the study aims to highlight the strengths and limitations of advanced machine learning techniques in financial forecasting and determine their practical viability.

The scope of this research is intentionally focused on short-term stock price predictions. While this allows for a controlled examination of machine learning methods, it excludes qualitative factors such as market sentiment, news events, and unforeseen macroeconomic shocks. Additionally, the analysis assumes the availability of high-quality historical data, which may not always be feasible in real-world applications. These limitations define the boundaries of the study while leaving room for future exploration of more comprehensive forecasting systems that integrate qualitative and quantitative factors.

By bridging the gap between traditional statistical methods and advanced machine learning techniques, this research contributes to the growing field of data-driven financial analysis. The findings aim to demonstrate the transformative potential of artificial intelligence in financial markets, providing insights into how machine learning can enhance the accuracy and reliability of stock price predictions. This study not only underscores the importance of integrating AI into financial decision-making but also lays the groundwork for future research that could expand the scope of these methods to include broader market dynamics and longer-term forecasting.

2. LITERATURE REVIEW

The intersection of machine learning (ML) and data analytics has revolutionized stock market prediction. While traditional statistical models like ARIMA have been used, advanced deep learning frameworks like LSTMs have shown superior performance in handling the complex and volatile nature of financial markets.

Recent studies have demonstrated the effectiveness of deep learning models in capturing sequential patterns and market volatility. Mohan et al. (2024) found that LSTM outperformed ARIMA, SARIMAX, and XGBoost when combined with sentiment-driven data. Similarly, Panchal et al. (2024) showed LSTM's superiority over ARIMA in managing sudden market fluctuations.

Sentiment analysis has emerged as a valuable tool for enhancing stock prediction. Rajhans et al. (2024) explored the impact of financial news sentiment on stock prices using CNN, GRU, and LSTM-GRU hybrid models. While their research highlighted the potential of these models, it also emphasized the need for larger and more diverse datasets to improve generalizability.

Optimizing model architecture is another crucial aspect. Taslim et al. (2024) investigated LSTM hyperparameter tuning and found that using SGDM optimizers and increasing training epochs significantly reduced RMSE. However, their study was limited by the use of a single dataset and inconclusive results regarding the optimal number of LSTM units.

Despite these advancements, challenges remain. Many studies are limited by data scope, reliance on specific stocks, and insufficient exploration of real-time data integration. Amruth et al. (2024) compared multiple machine learning models, finding XGBoost and Gradient Boost to perform well under certain conditions. However, their findings were limited by stock-specific factors and reliance on real-time data.

To address these limitations, this project aims to leverage advanced ML models for stock market analysis. By focusing on robust feature engineering, model optimization, and the integration of sentiment-driven data, this work aims to improve predictive accuracy and generalizability. By emphasizing metrics like RMSE and MSE, this project contributes to refining benchmarks for evaluating predictive performance.

3. OBJECTIVE

This study aims to develop a robust predictive framework for stock price forecasting in the Nifty 50 IT sector by leveraging machine learning models and event-driven data. The primary focus is on stock prices as the dependent variable, with predictors including daily returns, trading volume, and event-driven data such as economic announcements and global news. By implementing and comparing ARIMA, SARIMA, GRU, and LSTM models, the study seeks to assess predictive accuracy and demonstrate the advantage of RNNs in capturing complex market movements influenced by external factors.

The scope includes analyzing stock performance from 2006 to the present, focusing on major IT companies within the Nifty 50. The study identifies significant external events contributing to market fluctuations and evaluates model performance using metrics like RMSE, MAPE, and R-squared. To maintain clarity, the study avoids redundant variables and excludes minor events with negligible impact, ensuring that only strictly necessary data is considered. This approach systematically addresses the research problem while providing actionable insights into market dynamics.

4. PROPOSED METHODOLOGY

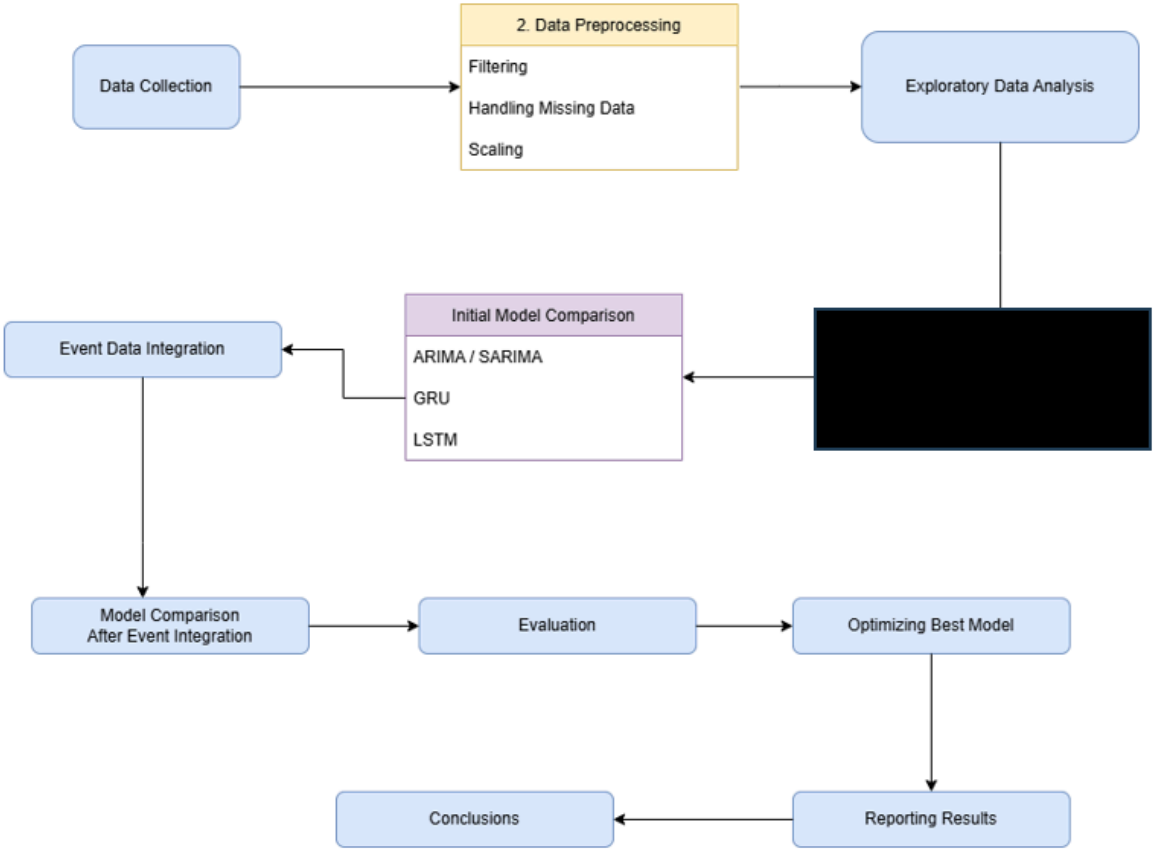


Fig 4.1 Methodology

1. Data Collection

The data for the companies under study was collected using the Yahoo Finance API. The data included stock prices of major Indian IT companies, namely Tata Consultancy Services, Infosys, Wipro, HCL Technologies, and Tech Mahindra. The historical stock data was fetched at a daily interval, spanning the entire duration for which Yahoo Finance provides data.

2. Data Preprocessing

The preprocessing phase involved several key steps to prepare the data for model training. First, the data was indexed by the date to ensure the time series structure. Any rows corresponding to the latest trading day were removed to ensure consistency in the dataset. Missing values due to holidays or non-trading days were

filled using forward-fill techniques. Additionally, any gaps in the data were handled by resampling the dataset to ensure daily data points. The stock price data was then scaled using standard scaling to normalize the features. A new aggregated column was created to represent the combined performance of all stocks, which was later used as a target variable for prediction.

3. Exploratory Data Analysis (EDA)

The EDA phase aimed to understand the patterns and characteristics of the data, as well as identify relationships between the various features. Time series plots of the stock prices were created to visualize their trends over time. Additionally, daily returns were calculated and plotted as percentage changes to assess the volatility of each stock. A correlation matrix was generated to observe the relationships between the stock prices of different companies, gold prices, the USD/INR forex rate, and the S&P 500 index. Finally, a 30-day rolling volatility plot was constructed to assess the stability and risk associated with each stock. The purpose of EDA was to identify potential patterns, outliers, and trends in the data, and to inform the choice of model features.

4. Event Data Integration

For the purpose of improving model accuracy, external event data such as gold prices, the USD/INR forex rate, and the S&P 500 index were integrated into the stock data. These external factors could potentially influence stock prices. The data for these external variables was collected, and it was merged with the stock data using the date as a common key. Data consistency was ensured by filtering, filling missing values, and resampling where necessary.

5. Preparing and Training Models

ARIMA Model

The ARIMA model was employed to model the stock price data. Before applying ARIMA, stationarity of the data was checked using the Augmented Dickey-Fuller (ADF) test, with the p-value being used to assess the null hypothesis of non-stationarity. Auto-ARIMA was used to find the optimal set of parameters (p, d,

q). The ARIMA model was trained, and predictions were made for the next 10 days. A line plot was used to compare the predicted values with the actual values for the last 10 days in the training dataset.

SARIMA Model

Similar to ARIMA, the SARIMA model was trained on the stock data after ensuring stationarity. Seasonal differences and lagged terms were incorporated to capture any seasonal trends in the data. The optimal parameters for the SARIMA model were determined, and the model was evaluated with predictions for the next 10 days.

GRU Model

For the GRU model, the data was first scaled using MinMax scaling to normalize the features. A sequence of past data points (60 days in this case) was used to predict future stock prices. The GRU model, which is an advanced version of Recurrent Neural Networks (RNNs), was then trained using this sequence of features, including stock prices and external event data. Early stopping was used to prevent overfitting, with the model being trained for 50 epochs. Once the model was trained, it was used to predict stock prices for the next 10 days, and the predictions were inverse-transformed back to their original scale.

LSTM Model

The LSTM model, another variant of RNN, was similarly trained to predict stock prices. The process was nearly identical to the GRU model, where the data was scaled, sequences were created, and the model was trained using 60-day windows of past data. Early stopping was implemented to prevent overfitting, and the LSTM model was trained over 50 epochs. The predictions made by the LSTM model were also inverse-transformed to get the actual stock prices.

6. Evaluation

The performance of each model was evaluated using two primary metrics: the R-squared (R^2) score and the Mean Squared Error (MSE). The R^2 score provides a measure of how well the model explains the variance in the data, while the MSE

indicates how closely the model's predictions align with the actual values. These metrics were computed for all models (ARIMA, SARIMA, GRU, and LSTM), and the models were compared based on their performance.

7. Optimizing Best Models

Optimization of the best-performing models involved fine-tuning the hyperparameters. For both ARIMA and SARIMA, parameter optimization was conducted by experimenting with different combinations of p, d, and q values. For the GRU and LSTM models, the learning rate, number of epochs, and batch size were adjusted to find the best configuration. Grid search and manual tuning were used to improve model accuracy, and the models were retrained with the optimized parameters.

8. Reporting Results

The results of the evaluation were compiled and compared across all models. It was observed that the RNN-based models (GRU and LSTM) outperformed the ARIMA and SARIMA models in terms of both R^2 and MSE. This was expected, as RNN models are better equipped to capture the complex, non-linear relationships present in time series data. The report includes visual comparisons of the model predictions versus actual values and discusses the implications of the results. The RNN models, particularly the GRU and LSTM, were found to provide the most accurate forecasts, which confirms their suitability for stock price prediction tasks.

5. TOOLS AND TECHNIQUES

yfinance (Yahoo Finance API):

yfinance is a Python library that allows easy access to financial data from Yahoo Finance. It was used to download historical stock price data and other financial indicators required for the analysis. This tool enables convenient integration of time-series data into pandas DataFrames for further manipulation and analysis.

Statsmodels:

statsmodels is a Python library for statistical modeling. The ARIMA and adfuller functions from this package were used to check for stationarity in the time series and to fit ARIMA models. ARIMA is employed for univariate time-series forecasting, while the adfuller test is used to check the stationarity of the series through the Augmented Dickey-Fuller test.

pmdarima:

pmdarima is a Python package that simplifies the process of selecting ARIMA model parameters. The auto_arima function from this package was used to automatically select the best set of parameters (p, d, q) for ARIMA models, which helped optimize the forecasting model and improve prediction accuracy.

Matplotlib:

matplotlib is a plotting library for Python that was used to generate visualizations such as time-series plots, correlation matrices, and model predictions. These visualizations helped in performing exploratory data analysis (EDA) and in presenting results, such as the comparison of model predictions with actual data.

Scikit-learn:

Scikit-learn is a powerful machine learning library used for various tasks including data preprocessing and model evaluation. It provided tools for scaling data (StandardScaler), evaluating model performance (using metrics like MSE, MAE, and R^2), and handling data transformations required during model building.

Keras (with TensorFlow backend):

Keras is a high-level neural networks API that runs on top of TensorFlow, used in this project to build and train deep learning models such as GRU (Gated Recurrent Units) and LSTM (Long Short-Term Memory) networks. These models were employed for time-series forecasting, capturing

sequential dependencies in the data. The layers like GRU, LSTM, and Dense were used to define the model architecture.

TensorFlow:

TensorFlow is an open-source deep learning framework developed by Google, serving as the backend for the Keras models. It provides tools for building, training, and evaluating deep learning models, specifically for time-series forecasting in this project.

Keras Callbacks:

The EarlyStopping callback from Keras was used during model training to monitor validation loss and prevent overfitting. This allows the model to stop training once the validation performance stops improving for a specified number of epochs, ensuring efficient training.

Statsmodels API:

statsmodels.api was used for additional statistical modeling tasks. It provides tools for fitting models, including time-series analysis, and is widely utilized for statistical tests and model diagnostics, complementing the functions provided by statsmodels.tsa for ARIMA-based forecasting.

6. IMPLEMENTATION

6.1 About the Dataset

The Data was collected using the Yahoo Finance API.

- Source of the Dataset: Yahoo Finance API
- No. of Observations: 5951 per company
- Column/Feature Details: Date, Ticker, Open, Close, Adj Close, High, Low, Volume, APS, Gold Price, USD to INR, S&P 500
- Details about the columns:
    - **Date**: Discrete (time-series feature)
    - Price-related columns (**Adj Close**, **Close**, **High**, **Low**, **Open**): Continuous
    - **Volume**: Discrete (countable values)

6.2. Data Cleaning and Preprocessing

The preprocessing phase of this project was critical in preparing the stock market data for analysis and modeling. Data was sourced for five Indian IT sector companies listed on the Nifty 50 index using the Yahoo Finance API. Each dataset included key columns such as open, close, high, low, adjusted close, and volume. To enable efficient querying and analysis, a new "ticker" column was introduced, and both the "date" and "ticker" were used as indices. This indexing ensured that the data could be accessed conveniently, company-wise, and date-wise.

Data filtering involved removing rows corresponding to earlier dates for which some stocks lacked data. Maintaining uniformity in the dataset was crucial for consistent analysis, and discrepancies in date ranges across companies were resolved through this filtering process. While a check for missing values showed no null entries, this did not account for missing data in the true sense, as gaps in trading days (such as holidays) meant certain dates were entirely absent from the dataset. There were also cases where the trading volume was recorded as zero, indicating non-trading days. To address these issues, forward-filling techniques were employed to populate missing values, and the dataset was resampled to ensure daily continuity. This step was essential, particularly for time-series modeling methods such as ARIMA and SARIMA, which rely on consistent, uninterrupted data.

To prepare the data for ARIMA modeling, a new column called the Aggregate Performance Score (APS) was created. This column represented the combined performance of all five companies and served as the target variable for prediction. For convenience, a reduced dataset containing only the "date" and "APS" columns was extracted. Stationarity of this new dataset was tested using the Augmented Dickey-Fuller (ADF) test to ensure its suitability for ARIMA/SARIMA modeling. The stock price features were scaled using standard scaling to normalize the data, a step that improved the

numerical stability of machine learning models. This scaling was extended to all relevant features, including external market indicators such as the gold price, USD-INR exchange rate, and S&P 500 index, which were later incorporated into the RNN models.

For the RNN-based models (GRU and LSTM), the dataset was aggregated further. Instead of retaining individual rows for each company, the dataset was transformed so that each date corresponded to a single row with aggregated values for columns such as open, close, high, low, and volume. This step ensured that the models could work on a unified dataset while preserving the time-series structure. Rescaled features and APS values were then combined into a single dataset, and sequences were created to capture temporal dependencies. The preprocessing phase thus laid a strong foundation for accurate modeling and forecasting.

6.3 Exploratory Data Analysis

6.3.1 Time Series Plot

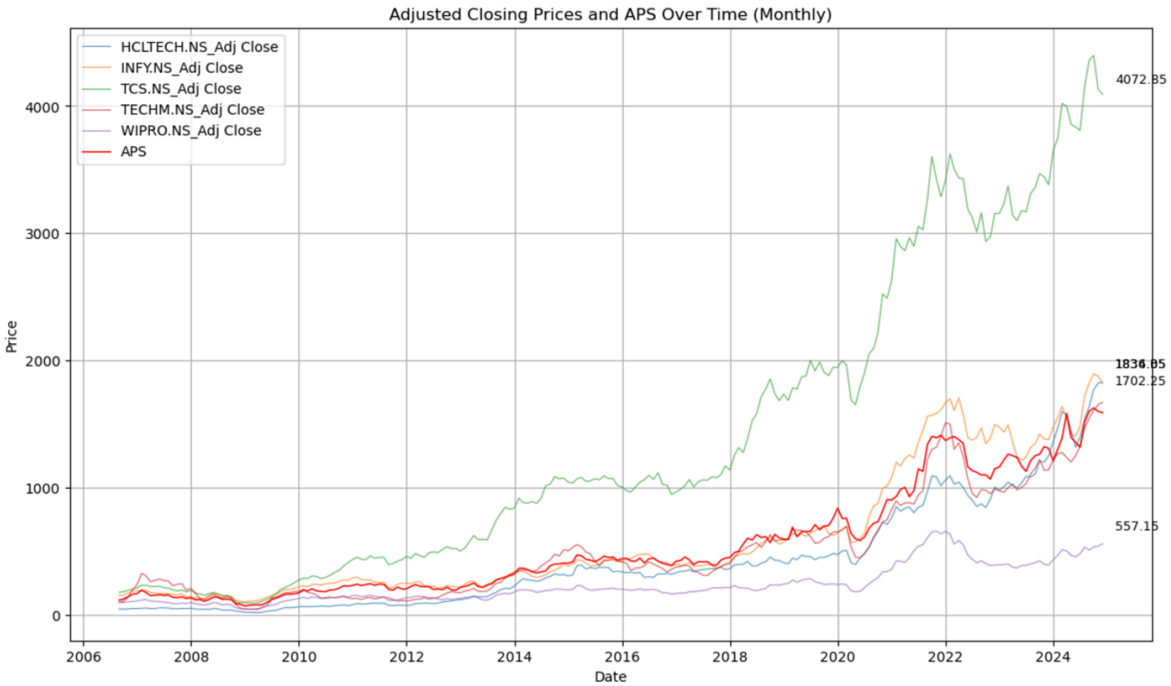


Fig 6.1 Time Series Plot

The time series plot shows an overall upward trend in the adjusted closing prices of major Indian IT companies, with significant growth after 2020 and notable divergence between companies. Key events include a dip during the 2008 financial crisis and a sharp drop and recovery during the COVID-19 pandemic.

6.3.2 Daily Returns Percentage Change Histograms

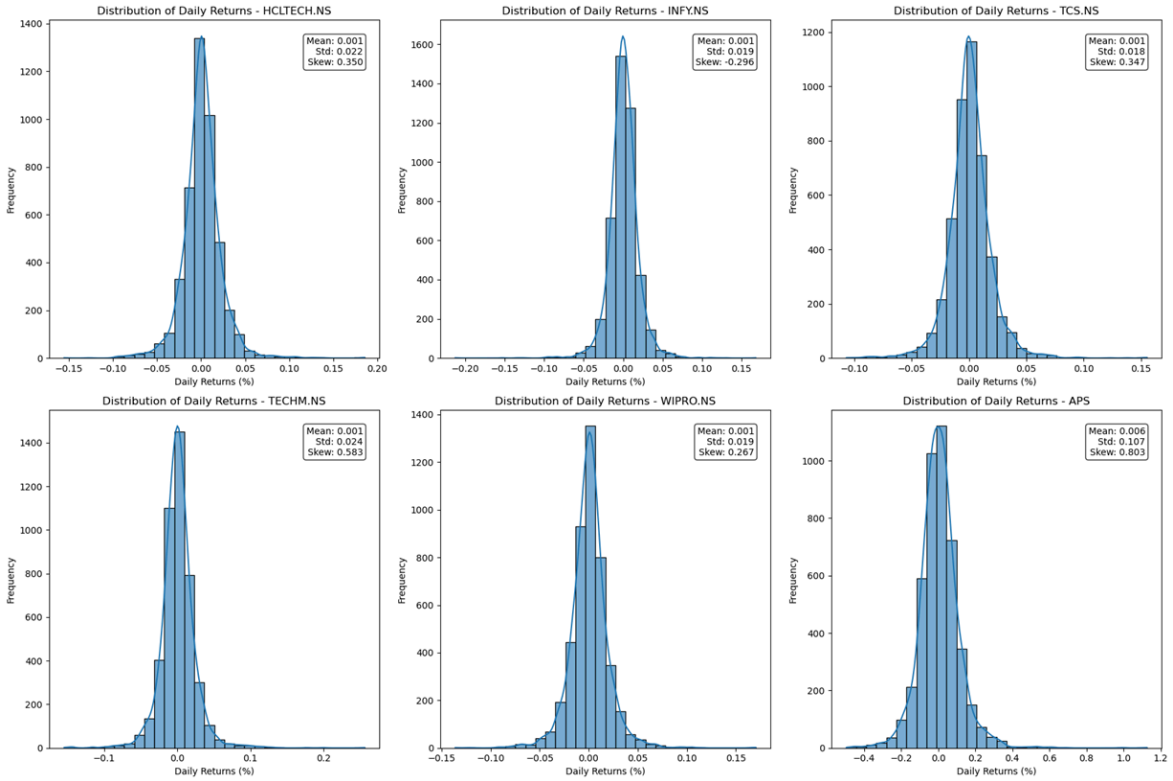


Fig 6.2 Daily Returns Percentage Change Histogram

The histograms of daily returns for all stocks show a bell-shaped distribution centered near zero, indicating that small daily price changes are most common, with rare extreme movements.

6.3.3 Correlation Matrix

Fig 6.3 Correlation Matrix

The correlation matrix shows moderate to strong positive relationships between the stocks of major Indian IT companies, with Infosys and Tata Consultancy Services exhibiting the highest correlation at 0.61.

6.3.4 30-Day Rolling Volatility Plot

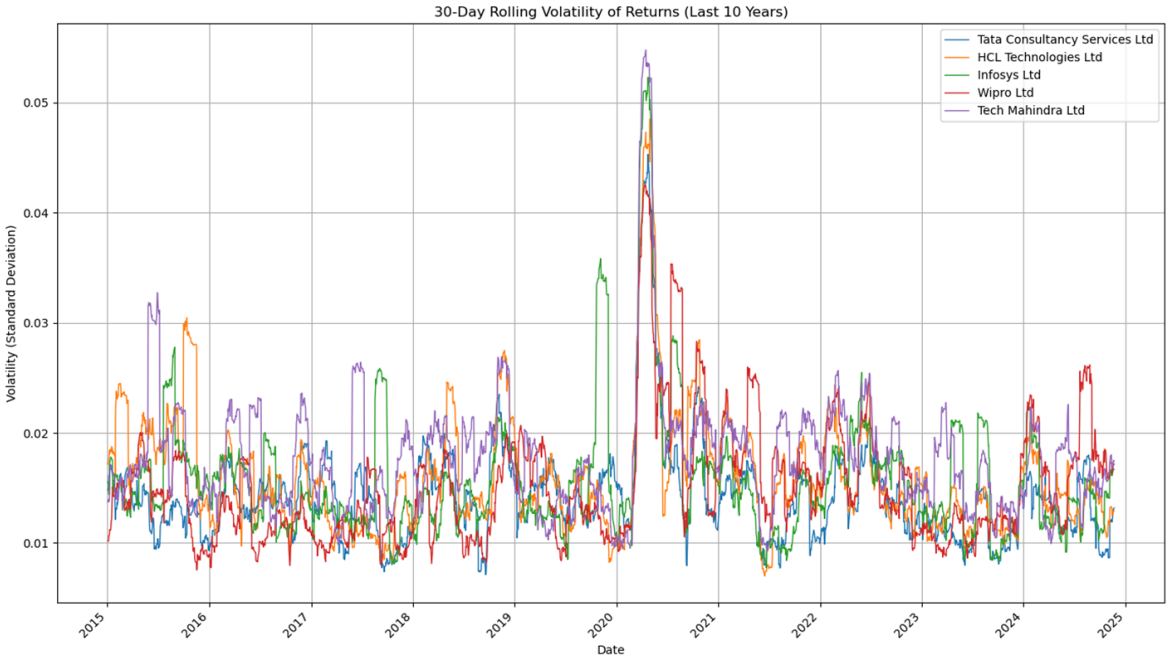


Fig 6.4 30-Day Rolling Volatility Plot

This 30-day rolling volatility plot highlights the uncertainty in the stock market around the time of the COVID-19 pandemic lockdowns.

6.4 Training Models and Predicting the future APS (Aggregate Performance Score)

The modeling phase involved three distinct approaches to forecast the future values of the Aggregate Performance Score (APS): ARIMA, SARIMA, and Recurrent Neural Networks (GRU and LSTM). Each method was tailored to capture different aspects of the time-series data.

ARIMA and SARIMA

For ARIMA modeling, the reduced dataset containing the “date” and “APS” columns was used. After verifying stationarity using the Augmented Dickey-Fuller test, the auto_arima function was employed to identify the optimal parameters (p, d, q) based on the Akaike Information Criterion (AIC). The best-fitting model was then trained, and a forecast for the next 10 days was generated. The predicted values were visualized on a line plot, comparing the last 10 observed days with the next 10 forecasted days, highlighting the model’s performance.

The SARIMA model extended this approach by incorporating seasonality into the analysis. Similar to ARIMA, auto_arima was used to determine the best seasonal parameters, and predictions were made for the same 10-day horizon. The results were again plotted to visualize the accuracy of the predictions in capturing seasonal trends.

GRU (Gated Recurrent Units)

To implement the GRU model, the fully aggregated dataset was used. This dataset contained scaled features for stock prices and external market indicators, as well as the scaled APS values. Temporal sequences were created using a sliding window approach, where each sequence consisted of 60 time steps corresponding to 60 days of historical data. The sequences were divided into input features (X) and target values (y). The data was then split into training and testing sets, with the latter constituting 20% of the data.

A GRU-based neural network was constructed with a single GRU layer containing 64 units, followed by a dense output layer with one unit to predict the APS value. The model was compiled using the Adam optimizer and a mean squared error loss function. Early stopping was implemented with a patience of five epochs to prevent overfitting and ensure that the model's weights were restored to the best-performing state. The training process involved 50 epochs and a batch size of 32.

Once the GRU model was trained, the last 60-day sequence from the test dataset was used as input to predict APS values for the next 10 days. Predictions were generated iteratively, with each new prediction incorporated back into the sequence for subsequent forecasts. The predicted values were then inverse-transformed to obtain the actual APS values. Finally, a line plot was created to compare the last 10 observed days with the next 10 predicted days, illustrating the model's forecasting ability.

LSTM (Long Short-Term Memory)

The LSTM model followed a similar process to the GRU model. The same aggregated dataset and scaled features were used to create temporal sequences. An LSTM-based neural network was designed with one LSTM layer containing 64 units, followed by a dense output layer with one unit. The model was compiled with the Adam optimizer and a mean squared error loss function. Early stopping was again utilized to enhance training efficiency and prevent overfitting.

After training the LSTM model using 50 epochs and a batch size of 32, predictions were made for the next 10 days using the last 60-day sequence from the test dataset. The iterative prediction approach was similar to the GRU implementation. Predicted APS values were inverse-transformed to their original scale and plotted alongside the last 10 observed days to visualize the forecast accuracy.

Both the GRU and LSTM models demonstrated the ability to capture complex patterns in the time-series data, with early stopping ensuring that the models generalized well to unseen data. The visual comparison of past and future APS values provided a clear assessment of the models' performance.

6.5 Evaluation

ARIMA

The ARIMA model was evaluated by splitting the dataset into training and testing sets using an 80-20 split. The model was trained on the first 80% of the data, and predictions were made for the remaining 20%. Evaluation metrics included Mean Squared Error (MSE) and R-squared. The ARIMA model produced an MSE of **42199.93**, indicating a substantial error in predictions. Furthermore, the R-squared value was **-0.198**, signifying that the model failed to capture the variance in the data and performed worse than a simple mean-based prediction.

SARIMA

Similar to ARIMA, the SARIMA model was trained and tested on an 80-20 split of the data. Seasonal parameters were optimized using an automated search. The model achieved an MSE of **75829.18**, which is higher than ARIMA, suggesting less accurate predictions. The R-squared value of **-1.153** highlights a severe underperformance, where the model failed to explain the variance in the testing data, likely due to overfitting or improper seasonal adjustments.

GRU

The GRU model utilized scaled input features and the APS target variable to predict stock prices. Data was split into training and testing sets, with sequences of 60 time steps used as input. The model was trained with early stopping to prevent overfitting. Evaluation metrics included MSE, RMSE, MAE, MAPE, and R-squared. The GRU model achieved an MSE of **18856.15**, significantly lower than ARIMA and SARIMA, demonstrating more accurate predictions. The R-squared value of **0.467** indicates that the GRU model explained nearly half of the variance in the data, outperforming the traditional statistical models.

LSTM

The LSTM model followed a similar process to the GRU, using scaled data and sequential inputs of 60 time steps. Evaluation metrics were consistent with those used for GRU. The LSTM model achieved an MSE of **20845.28**, slightly higher than the GRU model but still considerably lower than ARIMA and SARIMA. The R-squared value was **0.403**, reflecting that the LSTM model captured a significant portion of the variance in the data, albeit less effectively than the GRU model.

7. RESULTS AND DISCUSSIONS

The evaluation of ARIMA, SARIMA, GRU, and LSTM models revealed distinct performance differences in predicting stock market trends. The results, summarized in Table 1, indicate that deep learning models, particularly GRU and LSTM, outperformed traditional statistical methods like ARIMA and SARIMA. GRU achieved the highest R-squared value of 0.468 and the lowest Mean Squared Error (MSE) of 18,856.15, followed closely by LSTM with an R-squared of 0.403 and an MSE of 20,845.28. In contrast, ARIMA and SARIMA demonstrated poor predictive capabilities, with negative R-squared values and significantly higher MSEs, at 42,199.93 and 75,829.18, respectively.

Model	R-squared	Mean Squared Error
ARIMA	-0.19	42199.93
SARIMA	-1.15	75829.18
GRU	0.468	18856.15
LSTM	0.403	20845.28

Fig 7.1 Table comparing evaluation metrics of models

These findings highlight the limitations of ARIMA and SARIMA in capturing the non-linear dynamics inherent in stock market data, which is better modeled by deep learning techniques. GRU's superior performance can be attributed to its architecture, which effectively mitigates the vanishing gradient problem and excels in learning temporal dependencies from data. The observed patterns align with expectations, as the complex, non-linear nature of financial data is inherently more suitable for neural network-based models.

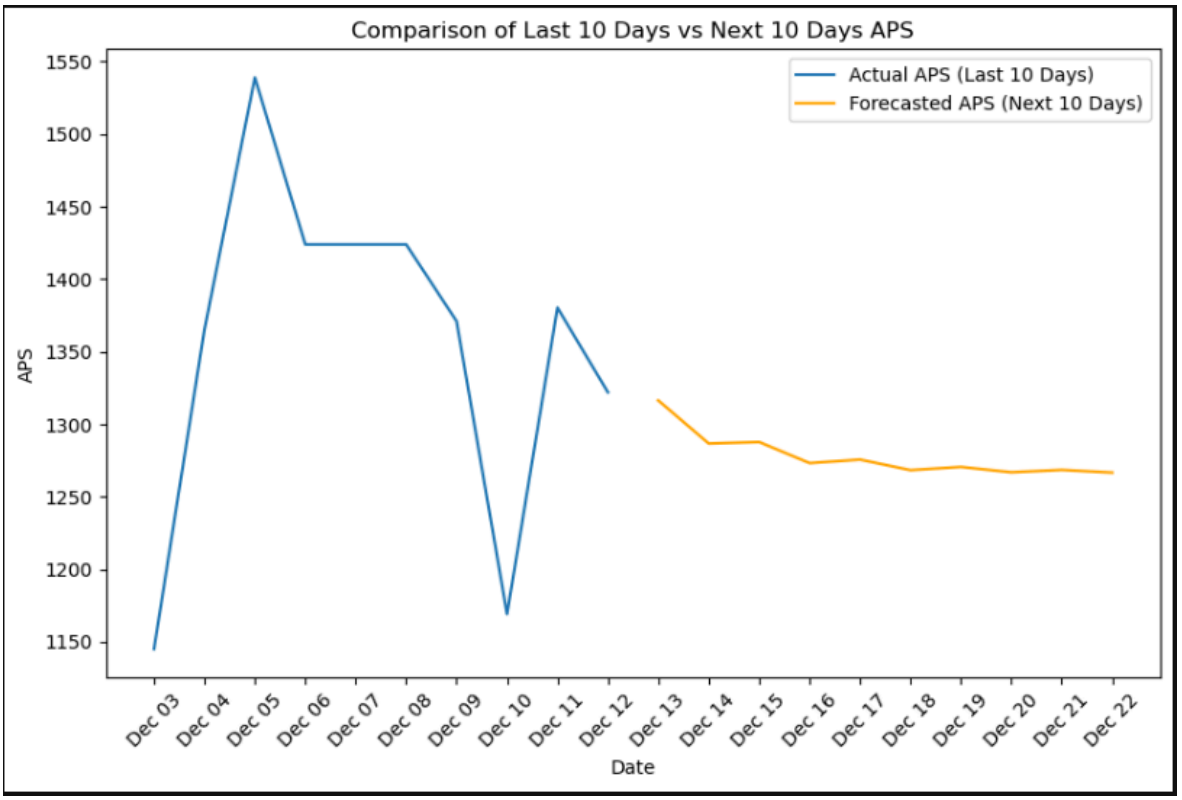


Fig 7.2 Last 10 days vs Predictions for Net 10 days (ARIMA)

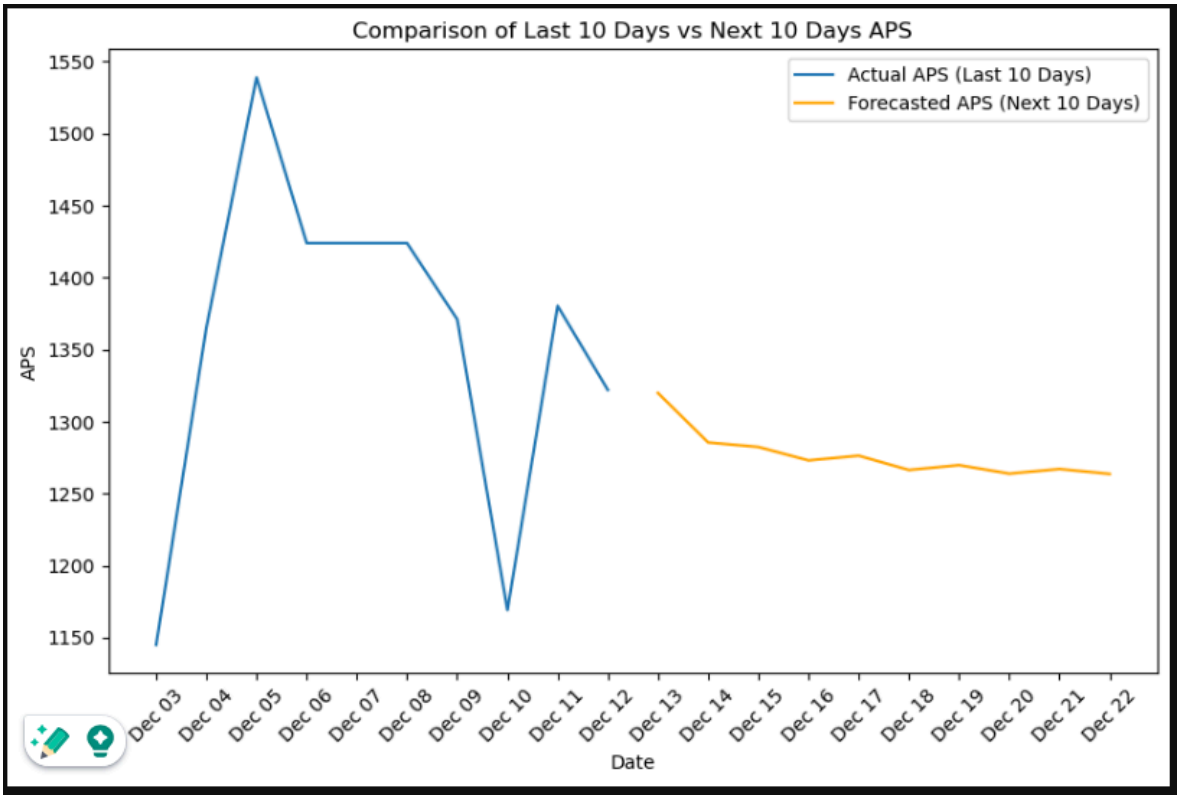


Fig 7.3 Last 10 days vs Predictions for Net 10 days (SARIMA)

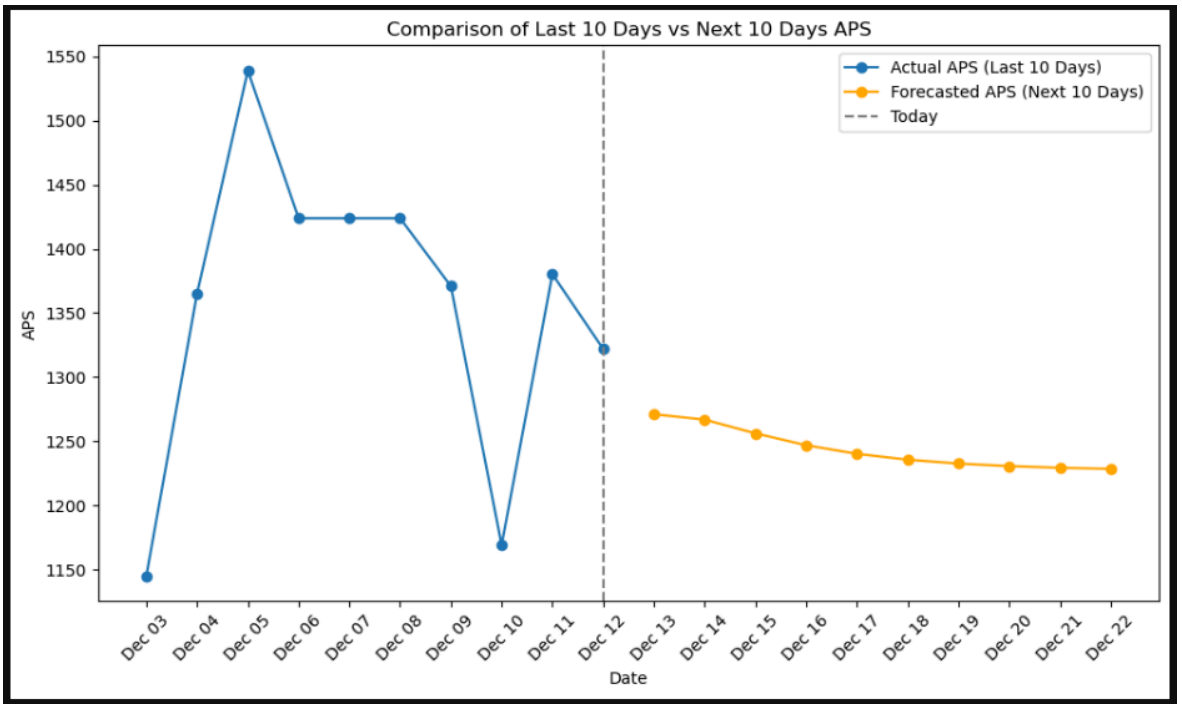


Fig 7.4 Last 10 days vs Predictions for Net 10 days (GRU)

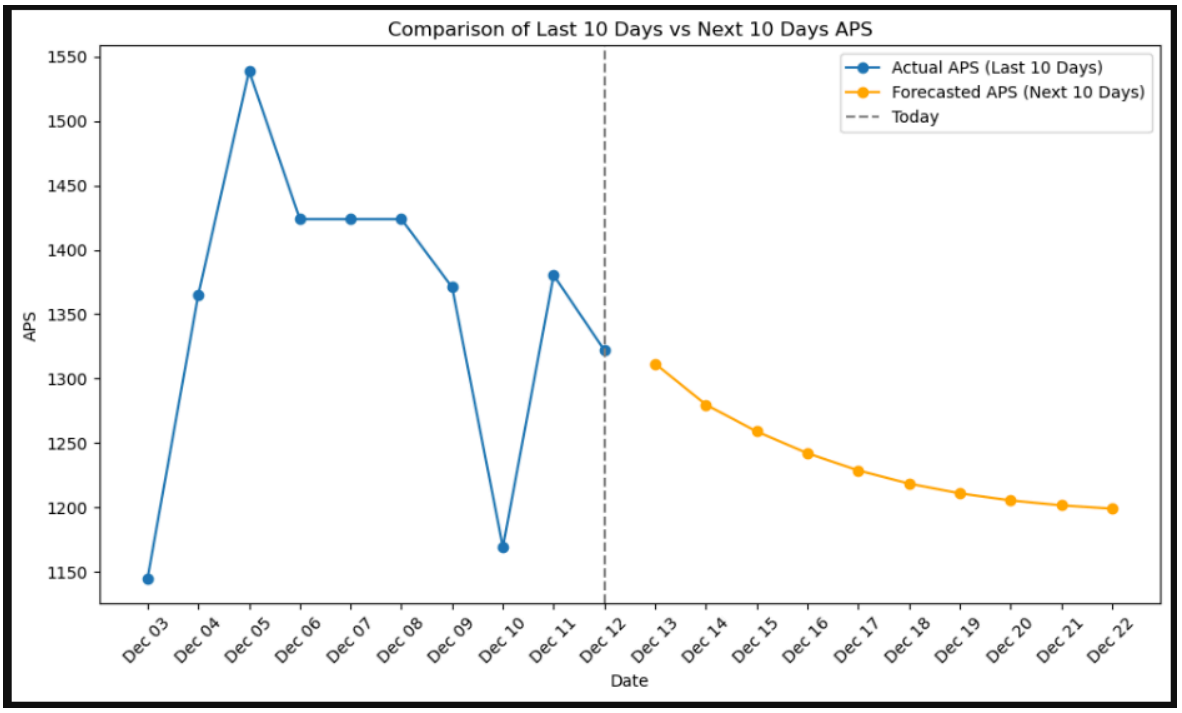


Fig 7.5 Last 10 days vs Predictions for Net 10 days (LSTM)

Comparing these results with related literature, while direct comparisons are limited by differences in datasets and methodologies, similar trends emerge. Studies consistently demonstrate the outperformance of deep learning models, such as LSTM and GRU, over traditional statistical methods. For instance, LSTM models in the literature achieved low

RMSEs across diverse financial datasets. However, the slight underperformance of LSTM in this study relative to GRU suggests the need for further tuning of hyperparameters or enhancements in feature engineering to fully leverage its potential. Overall, this study reaffirms the growing consensus on the efficacy of deep learning techniques for stock market forecasting.

8. CONCLUSION

This study highlights the capabilities and limitations of different predictive models in the context of stock market forecasting. Deep learning models such as GRU and LSTM proved to be more effective than traditional statistical methods like ARIMA and SARIMA, which struggled with the non-linear and volatile nature of stock price data. The success of GRU and LSTM underscores the importance of capturing temporal dependencies and complex patterns inherent in financial datasets. These models' ability to learn and adapt to intricate relationships within data marks a significant step forward for predictive analytics in finance.

At the core of these findings is the adaptability of neural networks in handling dynamic and unpredictable data, a task where statistical models falter. However, the results also reveal limitations, particularly in the context of scalability, hyperparameter tuning, and data requirements for deep learning models. These aspects highlight the need for further refinement and exploration to unlock their full potential.

The broader implications of this research extend to both academic and practical domains. By leveraging advanced machine learning techniques, stakeholders can develop more reliable tools for forecasting and risk management, ultimately aiding in investment decision-making. However, this work also serves as a reminder of the inherent uncertainties in financial markets, which no model can fully eliminate. Future efforts should focus on integrating external factors, such as economic indicators and news sentiment, into predictive frameworks to improve their robustness and applicability in real-world scenarios.

9. FUTURE ENHANCEMENT

Future work will focus on improving data preprocessing by exploring advanced techniques for handling missing values, such as imputation strategies that preserve data integrity. Building on the promising performance of GRU and LSTM, I plan to fine-tune their hyperparameters to optimize predictive accuracy further. Inspired by insights from related research, hybridizing GRU and LSTM models could leverage their complementary strengths, addressing challenges in capturing both short-term dependencies and long-term trends. Additionally, incorporating sentiment-driven analysis into the framework offers a compelling direction, as market sentiment often provides richer contextual signals for stock price movements compared to purely numerical data.

10. APPENDICES

10.1 FULL CODE

```
import warnings
warnings.filterwarnings("ignore")


import yfinance as yf
import pandas as pd
import os
import numpy as np


companies = {
    "Tata Consultancy Services": "TCS.NS",
    "Infosys": "INFY.NS",
    "Wipro": "WIPRO.NS",
    "HCL Technologies": "HCLTECH.NS",
    "Tech Mahindra": "TECHM.NS"
}

def fetch_and_save_data(ticker, period, interval, filename):
    # Download the data
    data = yf.download(ticker, period=period, interval=interval)

    # Save the data to a CSV file
    data.to_csv(filename)
    print(f"Data for {ticker} saved to {filename}")

    # Print the columns and the index of the data
    print(f"\nData columns for {ticker}:")
    print(data.columns)

    print(f"\nData index for {ticker}:")
    print(data.index)

# List to store the DataFrames for each company
dataframes = []

# Fetch data for each company
for company, ticker in companies.items():
    filename = f"{company}.csv"
    fetch_and_save_data(ticker, period="max", interval="1d",
```

```
filename=filename)

    # Load the data into a DataFrame
    data = pd.read_csv(filename, index_col=0, parse_dates=True)

    # Add a column to identify the company
    data['Ticker'] = ticker

    # Append to the list of DataFrames
    dataframes.append(data)

# Concatenate all DataFrames along rows
all_data = pd.concat(dataframes, axis=0)

# Reset the index to have a flat structure
all_data.reset_index(inplace=True)

# Print the first few rows of the combined DataFrame
print(all_data.head())
```

```
[*********************100%***********************]  1 of 1 completed
Data for TCS.NS saved to Tata Consultancy Services.csv

Data columns for TCS.NS:
MultiIndex([('Adj Close', 'TCS.NS'),
            (    'Close', 'TCS.NS'),
            (     'High', 'TCS.NS'),
            (      'Low', 'TCS.NS'),
            (     'Open', 'TCS.NS'),
            (   'Volume', 'TCS.NS')],
           names=['Price', 'Ticker'])

Data index for TCS.NS:
DatetimeIndex(['2002-08-12 00:00:00+00:00', '2002-08-13 00:00:00+00:00',
               '2002-08-14 00:00:00+00:00', '2002-08-15 00:00:00+00:00',
               '2002-08-16 00:00:00+00:00', '2002-08-19 00:00:00+00:00',
               '2002-08-20 00:00:00+00:00', '2002-08-21 00:00:00+00:00',
               '2002-08-22 00:00:00+00:00', '2002-08-23 00:00:00+00:00',
               ...
               '2024-11-29 00:00:00+00:00', '2024-12-02 00:00:00+00:00',
               '2024-12-03 00:00:00+00:00', '2024-12-04 00:00:00+00:00',
               '2024-12-05 00:00:00+00:00', '2024-12-06 00:00:00+00:00',
               '2024-12-09 00:00:00+00:00', '2024-12-10 00:00:00+00:00',
               '2024-12-11 00:00:00+00:00', '2024-12-12 00:00:00+00:00'],
              dtype='datetime64[ns, UTC]', name='Date', length=5545, freq=None)
```

```
# Identify the row indices of the "Ticker" and "Date" rows
ticker_row_index = all_data[all_data['Price'] == 'Ticker'].index
date_row_index = all_data[all_data['Price'] == 'Date'].index

# Combine the indices into a single list
rows_to_drop = ticker_row_index.union(date_row_index)
```

```python
# Drop the rows
data_cleaned = all_data.drop(rows_to_drop)


# Reset the index to start from 0
data_cleaned = data_cleaned.reset_index(drop=True)

# Rename the columns
data_cleaned = data_cleaned.rename(columns={'Price': 'Date'})

# Save or use the cleaned data
data_cleaned.to_csv("cleaned_data.csv", index=False)


# Print the missing counts before imputation
print("\nMissing values count:")
missing_counts_before = filtered_data.isnull().sum()
for column, count in missing_counts_before.items():
    print(f"{column}: {count} missing values")
```

```
Missing values count:
Date: 0 missing values
Adj Close: 0 missing values
Close: 0 missing values
High: 0 missing values
Low: 0 missing values
Open: 0 missing values
Volume: 0 missing values
Ticker: 0 missing values
```

```python
zero_volume_data = filtered_data[filtered_data['Volume'] == 0]

# Print the dates and corresponding tickers where volume was 0
for index, row in zero_volume_data.iterrows():
    print(f"Ticker: {row['Ticker']}, Date: {row['Date']}, Volume:
{row['Volume']}")
```

```
Ticker: TCS.NS, Date: 2008-11-27 00:00:00+00:00, Volume: 0
Ticker: TCS.NS, Date: 2009-04-30 00:00:00+00:00, Volume: 0
Ticker: TCS.NS, Date: 2009-10-13 00:00:00+00:00, Volume: 0
Ticker: TCS.NS, Date: 2014-04-24 00:00:00+00:00, Volume: 0
Ticker: TCS.NS, Date: 2014-10-15 00:00:00+00:00, Volume: 0
Ticker: INFY.NS, Date: 2008-11-27 00:00:00+00:00, Volume: 0
Ticker: INFY.NS, Date: 2009-04-30 00:00:00+00:00, Volume: 0
Ticker: INFY.NS, Date: 2009-10-13 00:00:00+00:00, Volume: 0
Ticker: INFY.NS, Date: 2014-04-24 00:00:00+00:00, Volume: 0
Ticker: INFY.NS, Date: 2014-10-15 00:00:00+00:00, Volume: 0
Ticker: WIPRO.NS, Date: 2008-11-27 00:00:00+00:00, Volume: 0
Ticker: WIPRO.NS, Date: 2009-04-30 00:00:00+00:00, Volume: 0
Ticker: WIPRO.NS, Date: 2009-10-13 00:00:00+00:00, Volume: 0
Ticker: WIPRO.NS, Date: 2014-04-24 00:00:00+00:00, Volume: 0
Ticker: WIPRO.NS, Date: 2014-10-15 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2007-06-01 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2008-11-27 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2009-04-30 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2009-08-18 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2009-08-19 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2009-08-20 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2009-08-21 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2009-10-13 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-01-20 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-01-21 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-01-22 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-01-25 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-01-27 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-01-28 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-01-29 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-02-01 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-02-02 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-02-03 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2010-02-04 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2014-04-24 00:00:00+00:00, Volume: 0
Ticker: HCLTECH.NS, Date: 2014-10-15 00:00:00+00:00, Volume: 0
Ticker: TECHM.NS, Date: 2008-11-27 00:00:00+00:00, Volume: 0
Ticker: TECHM.NS, Date: 2009-04-30 00:00:00+00:00, Volume: 0
Ticker: TECHM.NS, Date: 2009-10-13 00:00:00+00:00, Volume: 0
Ticker: TECHM.NS, Date: 2014-04-24 00:00:00+00:00, Volume: 0
Ticker: TECHM.NS, Date: 2014-10-15 00:00:00+00:00, Volume: 0
```

```
# List of specific dates to check
dates_to_check = ['2014-10-15', '2014-04-24', '2009-10-13',
'2009-04-30', '2008-11-27']

# Convert the list of date strings to datetime objects (naive)
dates_to_check = pd.to_datetime(dates_to_check)

# Convert the naive dates to timezone-aware (UTC) datetime objects for
comparison
dates_to_check = dates_to_check.tz_localize('UTC')
```

```
# Loop through each ticker and date, and forward fill the volume for
those specific dates
for ticker in ['INFY.NS', 'TCS.NS', 'WIPRO.NS', 'HCLTECH.NS',
'TECHM.NS']:
    for date_to_check in dates_to_check:
        # Filter the data for the specific ticker and date
        data_on_date = filtered_data[(filtered_data['Ticker'] ==
ticker) & (filtered_data['Date'] == date_to_check)]

        # Check if the volume is 0, and if so, forward fill it
        if not data_on_date.empty and data_on_date['Volume'].iloc[0]
== 0:
            # Get the previous day's data for forward filling
            previous_data = filtered_data[(filtered_data['Ticker'] ==
ticker) & (filtered_data['Date'] < date_to_check)].tail(1)
            if not previous_data.empty:
                forward_fill_value = previous_data['Volume'].iloc[0]
                # Update the volume for this date using forward
filling
                filtered_data.loc[data_on_date.index, 'Volume'] =
forward_fill_value

# Check if the changes were applied
zero_volume_count_after = filtered_data[filtered_data['Volume'] ==
0].shape[0]
print(f"Number of rows with volume 0 after forward fill:
{zero_volume_count_after}")
```

```
Number of rows with volume 0 after forward fill: 16
```

```
# List of specific dates to check for HCLTECH.NS
dates_to_check = [
    '2007-06-01', '2009-08-18', '2009-08-19', '2009-08-20',
'2009-08-21',
    '2010-01-20', '2010-01-21', '2010-01-22', '2010-01-25',
'2010-01-27',
    '2010-01-28', '2010-01-29', '2010-02-01', '2010-02-02',
'2010-02-03',
    '2010-02-04', '2010-02-06'
]

# Convert the list of date strings to datetime objects (naive)
dates_to_check = pd.to_datetime(dates_to_check)

# Convert the naive dates to timezone-aware (UTC) datetime objects for
```

```
comparison
dates_to_check = dates_to_check.tz_localize('UTC')

# Filter for the specific ticker 'HCLTECH.NS' and loop through each
date to check
for date_to_check in dates_to_check:
    # Filter the data for the specific ticker and date
    data_on_date = filtered_data[(filtered_data['Ticker'] ==
'HCLTECH.NS') & (filtered_data['Date'] == date_to_check)]

    # Check if the volume is 0, and if so, forward fill it
    if not data_on_date.empty and data_on_date['Volume'].iloc[0] == 0:
        # Get the previous day's data for forward filling
        previous_data = filtered_data[(filtered_data['Ticker'] ==
'HCLTECH.NS') & (filtered_data['Date'] < date_to_check)].tail(1)
        if not previous_data.empty:
            forward_fill_value = previous_data['Volume'].iloc[0]
            # Update the volume for this date using forward filling
            filtered_data.loc[data_on_date.index, 'Volume'] =
forward_fill_value
        else:
            # If there's no data prior to this date, you may choose to
leave it as NaN or fill with a default value (e.g., 0)
            filtered_data.loc[data_on_date.index, 'Volume'] = None  #
or 0 if you prefer to assume no trading activity

# Check if the changes were applied
zero_volume_count_after = filtered_data[filtered_data['Volume'] ==
0].shape[0]
print(f"Number of rows with volume 0 after forward fill:
{zero_volume_count_after}")
```

```
Number of rows with volume 0 after forward fill: 0
```

```
# Define the start and end date range
start_date = '2006-08-28'
end_date = '2024-12-12'

# Convert the start and end dates to datetime objects
start_date = pd.to_datetime(start_date)
end_date = pd.to_datetime(end_date)

# Generate a list of all dates within the range
all_dates = pd.date_range(start=start_date, end=end_date,
freq='D').date
```

```python
# Get the unique dates present in the data (ignoring the time and
timezone)
existing_dates = filtered_data['Date'].dt.date.unique()

# Find the dates that are missing
missing_dates = [date for date in all_dates if date not in
existing_dates]

# Filter out weekends from the missing dates
missing_non_weekends = [date for date in missing_dates if
date.weekday() < 5]  # 0=Monday, 4=Friday

print(f"Total number of missing dates (including weekends):
{len(missing_dates)}")
print(f"Total number of missing dates (excluding weekends):
{len(missing_non_weekends)}")
```

```
Total number of missing dates (including weekends): 2171
Total number of missing dates (excluding weekends): 265
```

```python
# Function to calculate APS
def calculate_aps(row, ticker_data):
    total_volume = ticker_data['Volume'].sum()  # Sum of volumes for
that date
    aps = 0
    for ticker in ticker_data['Ticker'].unique():
        ticker_data_filtered = ticker_data[ticker_data['Ticker'] ==
ticker]
        ticker_volume = ticker_data_filtered['Volume'].iloc[0]
        weight = ticker_volume / total_volume
        adj_close = ticker_data_filtered['Adj Close'].iloc[0]
        aps += adj_close * weight
    return aps

# Initialize a new column 'APS' in the filtered data
filtered_data['APS'] = None

# Iterate through each unique date in the filtered data
for date in filtered_data['Date'].unique():
    # Get all rows for the current date
    ticker_data = filtered_data[filtered_data['Date'] == date]

    # Apply the APS calculation to each row for this date
    for index, row in ticker_data.iterrows():
        aps = calculate_aps(row, ticker_data)
```

```
        filtered_data.at[index, 'APS'] = aps

# Print the first few rows of the updated DataFrame to check
print(filtered_data.head())
```

```
                       Date   Adj Close       Close        High         Low  \
0 2006-08-28 00:00:00+00:00  183.815109  245.274994  246.324997  242.074997
1 2006-08-29 00:00:00+00:00  185.501297  247.524994  248.000000  245.750000
2 2006-08-30 00:00:00+00:00  184.320938  245.949997  248.750000  245.262497
3 2006-08-31 00:00:00+00:00  186.990784  249.512497  251.000000  246.000000
4 2006-09-01 00:00:00+00:00  187.215561  249.812500  250.250000  247.774994

          Open   Volume  Ticker         APS
0  243.750000  2300316  TCS.NS  102.437767
1  246.250000  2679596  TCS.NS  105.166672
2  247.774994  2744772  TCS.NS   90.425672
3  246.000000  6076856  TCS.NS  103.671548
4  249.000000  1728728  TCS.NS  101.533377
```

```
start_date = "2006-08-28"
end_date = "2024-12-13"

# Fetch Gold Futures data in USD (Ticker: GC=F)
gold_data = yf.download("GC=F", start=start_date, end=end_date)

# Fetch USD to INR exchange rate data (Ticker: USDINR=X)
usd_inr_data = yf.download("USDINR=X", start=start_date, end=end_date)

# Select and rename the relevant columns
gold_data = gold_data[['Adj Close']].rename(columns={'Adj Close':
'Gold Price (USD)'})
usd_inr_data = usd_inr_data[['Adj Close']].rename(columns={'Adj
Close': 'USD to INR'})

# Merge the two datasets on date (index)
combined_data = pd.merge(gold_data, usd_inr_data, left_index=True,
right_index=True, how='inner')


gold_price_usd = combined_data['Gold Price (USD)'].squeeze()  #
Convert to Series if DataFrame
usd_to_inr = combined_data['USD to INR'].squeeze()  # Convert to
Series if DataFrame

# Now multiply the Series
combined_data['Gold Price (INR)'] = gold_price_usd * usd_to_inr
```

```
combined_data.to_csv('gold_price_in_inr.csv')
gold_inr_data = pd.read_csv('gold_price_in_inr.csv', index_col=0)

# Reset the index to flatten the multi-index structure
gold_inr_data_reset = gold_inr_data.reset_index()

# Drop the first two rows, which are essentially the unwanted header
rows
gold_inr_data_reset = gold_inr_data_reset.drop([0, 1], axis=0)

gold_inr_data_cleaned = gold_inr_data_reset.drop(['Gold Price (USD)',
'USD to INR'], axis=1)
gold_inr_data_cleaned.rename(columns={'Price': 'Date'}, inplace=True)
gold_inr_data_cleaned.reset_index(drop=True, inplace=True)
gold_inr_data_cleaned['Date'] =
pd.to_datetime(gold_inr_data_cleaned['Date'])


usd_inr_df = usd_to_inr.reset_index()
usd_inr_df.columns = ['Date', 'USD to INR']

# Make sure the 'Date' column is of datetime type
usd_inr_df['Date'] = pd.to_datetime(usd_inr_df['Date'])


merged_data = pd.merge(gold_merged_data, usd_inr_df[['Date', 'USD to
INR']], on='Date', how='inner')


sp500_data = yf.download('^GSPC', start='2008-08-28',
end='2024-12-13')

# We only need the 'Adj Close' price for the S&P 500
sp500_data = sp500_data[['Adj Close']]

# Reset index and rename columns
sp500_data.reset_index(inplace=True)
sp500_data.rename(columns={'Date': 'Date', 'Adj Close': 'S&P 500'},
inplace=True)


sp500_data.to_csv('sp500.csv')

sp500_dt = pd.read_csv('sp500.csv', index_col=0)

# Reset the index to flatten the multi-index structure
sp500_dt_reset = sp500_dt.reset_index()
```

```
# Drop the first two rows, which are essentially the unwanted header
rows
sp500_dt_reset = sp500_dt_reset.drop([0], axis=0)

sp500_dt_cleaned = sp500_dt_reset.drop(['Price'], axis=1)
sp500_dt_cleaned.reset_index(drop=True, inplace=True)
sp500_dt_cleaned['Date'] = pd.to_datetime(sp500_dt_cleaned['Date'])

final_data = pd.merge(merged_data, sp500_dt_cleaned[['Date', 'S&P
500']], on='Date', how='inner')


final_data['Date'] = pd.to_datetime(final_data['Date'])
final_data.set_index('Date', inplace=True)

# Step 2: Create a full date range
full_date_range = pd.date_range(start=final_data.index.min(),
end=final_data.index.max(), freq='D')

# Step 3: Create a MultiIndex for the DataFrame, including the
'Ticker' column
tickers = final_data['Ticker'].unique()  # Assuming 'Ticker' column
contains all company tickers
multi_index = pd.MultiIndex.from_product([full_date_range, tickers],
names=['Date', 'Ticker'])

# Step 4: Reindex the DataFrame to include all dates and tickers
final_data_reindexed = final_data.reset_index().set_index(['Date',
'Ticker']).reindex(multi_index)


# Drop the unnamed column (which is the old range index)
final_data_reindexed = final_data_reindexed.loc[:,
~final_data_reindexed.columns.str.contains('^Unnamed')]
```

```
final_data_reindexed.tail()
```

		Adj Close	Close	High	Low	Open	Volume	APS	Gold Price (INR)	USD to INR	S&P 500
Date	Ticker										
2024-12-12 00:00:00+00:00	TCS.NS	4454.950195	4454.950195	4485.000000	4426.000000	4430.049805	1914617.0	1321.980742	229522.542132	84.822998	6076.549805
	INFY.NS	1987.000000	1987.000000	1998.800049	1970.550049	1975.199951	6461993.0	1321.980742	229522.542132	84.822998	6076.549805
	WIPRO.NS	309.100006	309.100006	313.799988	307.450012	308.000000	12956619.0	1321.980742	229522.542132	84.822998	6076.549805
	HCLTECH.NS	1936.199951	1936.199951	1949.000000	1921.500000	1924.949951	2306151.0	1321.980742	229522.542132	84.822998	6076.549805
	TECHM.NS	1789.599976	1789.599976	1807.699951	1761.050049	1768.050049	3017928.0	1321.980742	229522.542132	84.822998	6076.549805

```
final_data_filled = final_data_reindexed.ffill()

final_data_filled.index = final_data_filled.index.set_levels(
```

```
    pd.DatetimeIndex(final_data_filled.index.levels[0], freq='D'),
    level=0
)

# Check frequency
final_data_filled.index.levels[0].freq


from sklearn.preprocessing import StandardScaler

# Step 1: Reset index to access Ticker column easily
final_data_reset = final_data_filled.reset_index()

# Step 2: Initialize a new column for scaled values
final_data_reset['Adj Close Scaled'] = 0.0

# Step 3: Scale 'Adj Close' for each company independently
scaler = StandardScaler()

for ticker in final_data_reset['Ticker'].unique():
    # Filter data for the specific company
    company_data = final_data_reset[final_data_reset['Ticker'] ==
ticker]

    # Fit and transform the scaler for the company's Adj Close prices
    scaled_values = scaler.fit_transform(company_data[['Adj Close']])

    # Assign the scaled values back to the corresponding rows
    final_data_reset.loc[final_data_reset['Ticker'] == ticker, 'Adj
Close Scaled'] = scaled_values.flatten()

# Step 4: Save the scaled data into a new DataFrame with the same
structure
final_data_scaled = final_data_reset.set_index(['Date', 'Ticker'])


final_data_scaled.index = final_data_scaled.index.set_levels(
    pd.DatetimeIndex(final_data_scaled.index.levels[0], freq='D'),
    level=0
)

# Check frequency
final_data_scaled.index.levels[0].freq
```

<Day>

final_data_scaled

Date	Ticker	Adj Close	Adj Close Scaled	Close	High	Low	Open	Volume	APS	Gold Price (INR)	USD to INR	S&P 5
2008-08-28 00:00:00+00:00	TCS.NS	152.424805	-0.957027	198.087494	206.062500	195.662506	205.000000	6580688.0	90.334791	36268.787334	43.650002	1300.680
	INFY.NS	150.438522	-0.970708	212.425003	214.625000	212.000000	214.125000	11480800.0	90.334791	36268.787334	43.650002	1300.680
	WIPRO.NS	37.649399	-0.725630	46.586250	48.588753	45.939377	48.037502	16700896.0	90.334791	36268.787334	43.650002	1300.680
	HCLTECH.NS	38.026783	-1.081587	56.000000	58.750000	55.287498	58.737499	4692348.0	90.334791	36268.787334	43.650002	1300.680
	TECHM.NS	137.753387	-0.986049	184.175003	195.975006	182.812500	193.750000	563212.0	90.334791	36268.787334	43.650002	1300.680
...	...	...	...	...	...	...	...	...	...	...	...	
2024-12-12 00:00:00+00:00	TCS.NS	4454.950195	2.889315	4454.950195	4485.000000	4426.000000	4430.049805	1914617.0	1321.980742	229522.542132	84.822998	6076.549
	INFY.NS	1987.000000	2.853294	1987.000000	1998.800049	1970.550049	1975.199951	6461993.0	1321.980742	229522.542132	84.822998	6076.549
	WIPRO.NS	309.100006	0.127851	309.100006	313.799988	307.450012	308.000000	12956619.0	1321.980742	229522.542132	84.822998	6076.549
	HCLTECH.NS	1936.199951	3.383610	1936.199951	1949.000000	1921.500000	1924.949951	2306151.0	1321.980742	229522.542132	84.822998	6076.549
	TECHM.NS	1789.599976	3.038617	1789.599976	1807.699951	1761.050049	1768.050049	3017928.0	1321.980742	229522.542132	84.822998	6076.549

29755 rows × 11 columns

```
import pmdarima as pm


def check_stationarity(series):
    result = adfuller(series)
    print('ADF Statistic:', result[0])
    print('p-value:', result[1])

check_stationarity(aps_df['APS'])
```

```
ADF Statistic: -0.5131378080913502
p-value: 0.8894271426108918
```

```
# Step 2: Use AutoARIMA to find the best ARIMA parameters (p, d, q)
arima_model = pm.auto_arima(aps_df['APS'], seasonal=False,
stepwise=True, trace=True, error_action='ignore',
suppress_warnings=True)

# Print the best ARIMA model parameters
print(f"Best ARIMA model: {arima_model.summary()}")
```

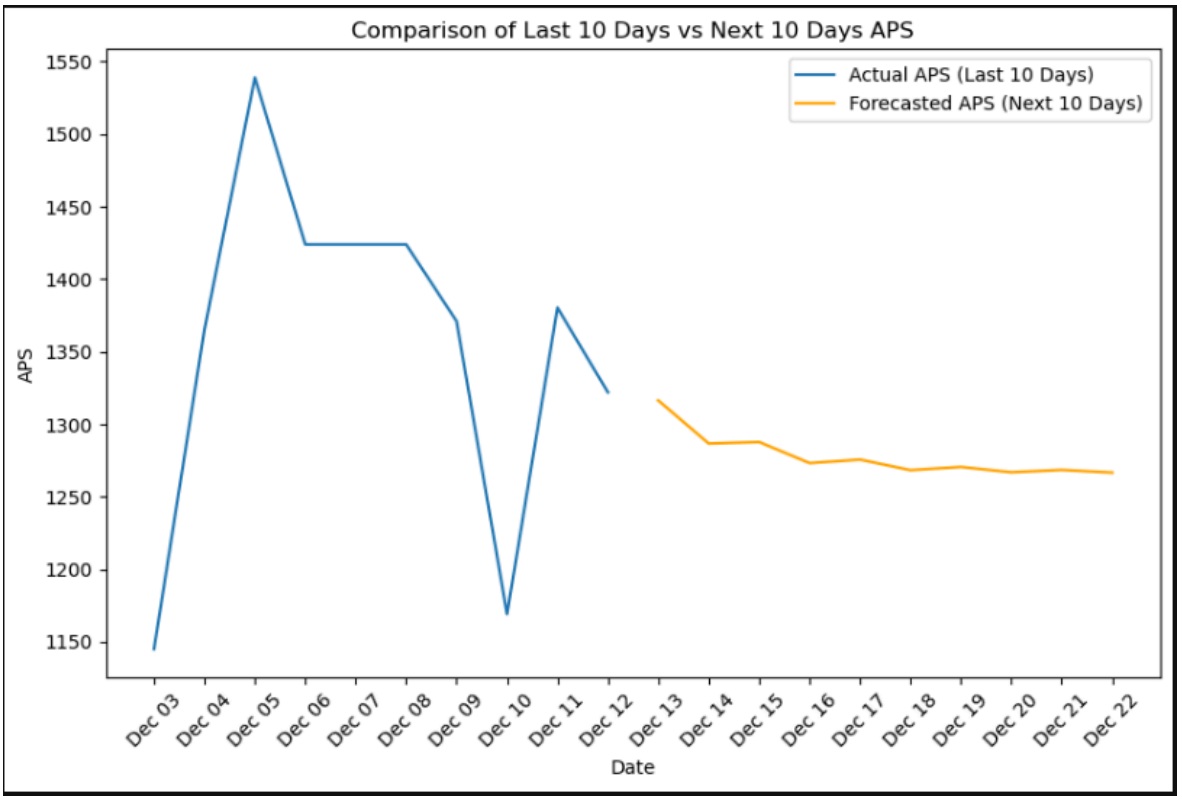
```
Performing stepwise search to minimize aic
 ARIMA(2,1,2)(0,0,0)[0] intercept   : AIC=67463.468, Time=6.16 sec
 ARIMA(0,1,0)(0,0,0)[0] intercept   : AIC=68573.372, Time=0.20 sec
 ARIMA(1,1,0)(0,0,0)[0] intercept   : AIC=68206.687, Time=0.43 sec
 ARIMA(0,1,1)(0,0,0)[0] intercept   : AIC=68057.774, Time=0.70 sec
 ARIMA(0,1,0)(0,0,0)[0]             : AIC=68571.415, Time=0.08 sec
 ARIMA(1,1,2)(0,0,0)[0] intercept   : AIC=67486.711, Time=4.02 sec
 ARIMA(2,1,1)(0,0,0)[0] intercept   : AIC=67484.755, Time=4.36 sec
 ARIMA(3,1,2)(0,0,0)[0] intercept   : AIC=67465.468, Time=7.34 sec
 ARIMA(2,1,3)(0,0,0)[0] intercept   : AIC=67465.468, Time=7.49 sec
 ARIMA(1,1,1)(0,0,0)[0] intercept   : AIC=67507.233, Time=2.75 sec
 ARIMA(1,1,3)(0,0,0)[0] intercept   : AIC=67485.313, Time=6.60 sec
 ARIMA(3,1,1)(0,0,0)[0] intercept   : AIC=67481.735, Time=5.20 sec
 ARIMA(3,1,3)(0,0,0)[0] intercept   : AIC=67467.132, Time=10.69 sec
 ARIMA(2,1,2)(0,0,0)[0]             : AIC=67465.833, Time=2.41 sec

Best model:  ARIMA(2,1,2)(0,0,0)[0] intercept
Total fit time: 58.445 seconds
Best ARIMA model:                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                 5951
Model:               SARIMAX(2, 1, 2)   Log Likelihood              -33725.734
Date:                Fri, 13 Dec 2024   AIC                          67463.468
Time:                        15:08:18   BIC                          67503.615
Sample:                    08-28-2008   HQIC                         67477.416
                         - 12-12-2024
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
intercept      0.1248      0.061      2.043      0.041       0.005       0.244
ar.L1         -0.1234      0.042     -2.973      0.003      -0.205      -0.042
ar.L2          0.4889      0.021     22.917      0.000       0.447       0.531
ma.L1         -0.2619      0.043     -6.081      0.000      -0.346      -0.177
ma.L2         -0.6739      0.040    -16.864      0.000      -0.752      -0.596
sigma2      4906.5281     30.942    158.573      0.000    4845.883    4967.173
===================================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):             88911.34
Prob(Q):                              0.99   Prob(JB):                         0.00
Heteroskedasticity (H):              37.32   Skew:                            -0.78
Prob(H) (two-sided):                  0.00   Kurtosis:                        21.87
===================================================================================
```

```
# Step 3: Forecast the next 10 days using the best model
forecast_steps = 10
forecast = arima_model.predict(n_periods=forecast_steps)

# Create a DateTime index for the next 10 days
last_date = aps_df.index[-1]
forecast_dates = pd.date_range(last_date, periods=forecast_steps + 1,
freq='D')[1:]

# Create a DataFrame for the forecast results
forecast_df = pd.DataFrame({'Date': forecast_dates, 'Forecasted_APS':
```

```
forecast})
forecast_df.set_index('Date', inplace=True)

# Print the forecast
print(forecast_df)
```

```
                           Forecasted_APS
Date
2024-12-13 00:00:00+00:00     1316.366542
2024-12-14 00:00:00+00:00     1286.647634
2024-12-15 00:00:00+00:00     1287.695132
2024-12-16 00:00:00+00:00     1273.160377
2024-12-17 00:00:00+00:00     1275.591031
2024-12-18 00:00:00+00:00     1268.309451
2024-12-19 00:00:00+00:00     1270.521236
2024-12-20 00:00:00+00:00     1266.812905
2024-12-21 00:00:00+00:00     1268.476701
2024-12-22 00:00:00+00:00     1266.583039
```

```
# Step 4: Plot the actual data and forecasted values
# Get the last 10 days from the actual data
last_10_days = aps_df.tail(10)

# Combine the last 10 days of actual data with the forecasted values
combined_df = pd.concat([last_10_days, forecast_df])

plt.figure(figsize=(10, 6))
plt.plot(last_10_days.index, last_10_days['APS'], label='Actual APS
(Last 10 Days)')
plt.plot(forecast_df.index, forecast_df['Forecasted_APS'],
label='Forecasted APS (Next 10 Days)', color='orange')
plt.xlabel('Date')
plt.ylabel('APS')
plt.title('Comparison of Last 10 Days vs Next 10 Days APS')

# Format the x-axis labels: rotate and only show month and day
plt.xticks(ticks=combined_df.index, labels=[label.strftime('%b %d')
for label in combined_df.index], rotation=45)

plt.legend()
plt.show()
```




```
train_size = int(len(aps_df) * 0.8)
train_data, test_data = aps_df[:train_size], aps_df[train_size:]

# Fit the ARIMA model to the training data
model = sm.tsa.ARIMA(train_data['APS'], order=(2, 1, 2))  # Replace
(p, d, q) with optimal values
model_fit = model.fit()

# Make predictions on the test data
forecast = model_fit.forecast(steps=len(test_data))

# Calculate MSE
mse = mean_squared_error(test_data['APS'], forecast)
print('Mean Squared Error:', mse)

# Calculate R-squared
y_true = test_data['APS']
y_pred = forecast
y_mean = np.mean(y_true)

# Calculate the squared residuals
ssr = np.sum((y_true - y_pred) ** 2)

# Calculate the total squared deviations from the mean
sst = np.sum((y_true - y_mean) ** 2)
```

```
# Calculate R-squared
r_squared = 1 - ssr / sst
print('R-squared:', r_squared)
```

```
Mean Squared Error: 42199.932282940266
R-squared: -0.19799535088276787
```

```
# Step 1: Use AutoSARIMA to find the best seasonal ARIMA parameters
(p, d, q) and seasonal order (P, D, Q, s)
# s is the number of periods in each season, for daily data, s could
be 5 if we think there's weekly seasonality
sarima_model = pm.auto_arima(aps_df['APS'], seasonal=True, m=5,
stepwise=True, trace=True, error_action='ignore',
suppress_warnings=True)

# Print the best SARIMA model parameters
print(f"Best SARIMA model: {sarima_model.summary()}")
```

```
Performing stepwise search to minimize aic
 ARIMA(2,1,2)(1,0,1)[5] intercept   : AIC=67460.237, Time=12.99 sec
 ARIMA(0,1,0)(0,0,0)[5] intercept   : AIC=68573.372, Time=0.18 sec
 ARIMA(1,1,0)(1,0,0)[5] intercept   : AIC=68203.928, Time=1.78 sec
 ARIMA(0,1,1)(0,0,1)[5] intercept   : AIC=68038.529, Time=2.30 sec
 ARIMA(0,1,0)(0,0,0)[5]             : AIC=68571.415, Time=0.08 sec
 ARIMA(2,1,2)(0,0,1)[5] intercept   : AIC=67458.389, Time=9.38 sec
 ARIMA(2,1,2)(0,0,0)[5] intercept   : AIC=67463.468, Time=5.81 sec
 ARIMA(2,1,2)(0,0,2)[5] intercept   : AIC=67460.048, Time=15.92 sec
 ARIMA(2,1,2)(1,0,0)[5] intercept   : AIC=67458.292, Time=7.33 sec
 ARIMA(2,1,2)(2,0,0)[5] intercept   : AIC=67460.170, Time=14.03 sec
 ARIMA(2,1,2)(2,0,1)[5] intercept   : AIC=67461.899, Time=21.76 sec
 ARIMA(1,1,2)(1,0,0)[5] intercept   : AIC=67488.456, Time=6.74 sec
 ARIMA(2,1,1)(1,0,0)[5] intercept   : AIC=67486.357, Time=6.11 sec
 ARIMA(3,1,2)(1,0,0)[5] intercept   : AIC=67460.140, Time=12.38 sec
 ARIMA(2,1,3)(1,0,0)[5] intercept   : AIC=67460.118, Time=9.30 sec
 ARIMA(1,1,1)(1,0,0)[5] intercept   : AIC=67508.702, Time=5.04 sec
 ARIMA(1,1,3)(1,0,0)[5] intercept   : AIC=67485.968, Time=8.84 sec
 ARIMA(3,1,1)(1,0,0)[5] intercept   : AIC=67482.192, Time=8.46 sec
 ARIMA(3,1,3)(1,0,0)[5] intercept   : AIC=67460.305, Time=14.18 sec
 ARIMA(2,1,2)(1,0,0)[5]             : AIC=67460.864, Time=3.53 sec

Best model:  ARIMA(2,1,2)(1,0,0)[5] intercept
Total fit time: 166.240 seconds
Best SARIMA model:                                      SARIMAX Results
==========================================================================================
Dep. Variable:                                   y   No. Observations:                 5951
Model:             SARIMAX(2, 1, 2)x(1, 0, [], 5)   Log Likelihood              -33722.146
Date:                            Fri, 13 Dec 2024   AIC                           67458.292
Time:                                    15:17:07   BIC                           67505.130
Sample:                                08-28-2008   HQIC                          67474.564
                                     - 12-12-2024
Covariance Type:                              opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
intercept      0.1237      0.059      2.090      0.037       0.008       0.240
ar.L1         -0.1729      0.034     -5.017      0.000      -0.240      -0.105
ar.L2          0.5182      0.017     30.081      0.000       0.484       0.552
ma.L1         -0.2156      0.036     -6.044      0.000      -0.285      -0.146
ma.L2         -0.7223      0.033    -21.625      0.000      -0.788      -0.657
ar.S.L5        0.0383      0.009      4.041      0.000       0.020       0.057
sigma2      4900.5500     30.928    158.451      0.000    4839.933    4961.168
===================================================================================
Ljung-Box (L1) (Q):                   0.02   Jarque-Bera (JB):             88524.82
Prob(Q):                              0.88   Prob(JB):                         0.00
Heteroskedasticity (H):              37.16   Skew:                            -0.78
Prob(H) (two-sided):                  0.00   Kurtosis:                        21.83
===================================================================================
```

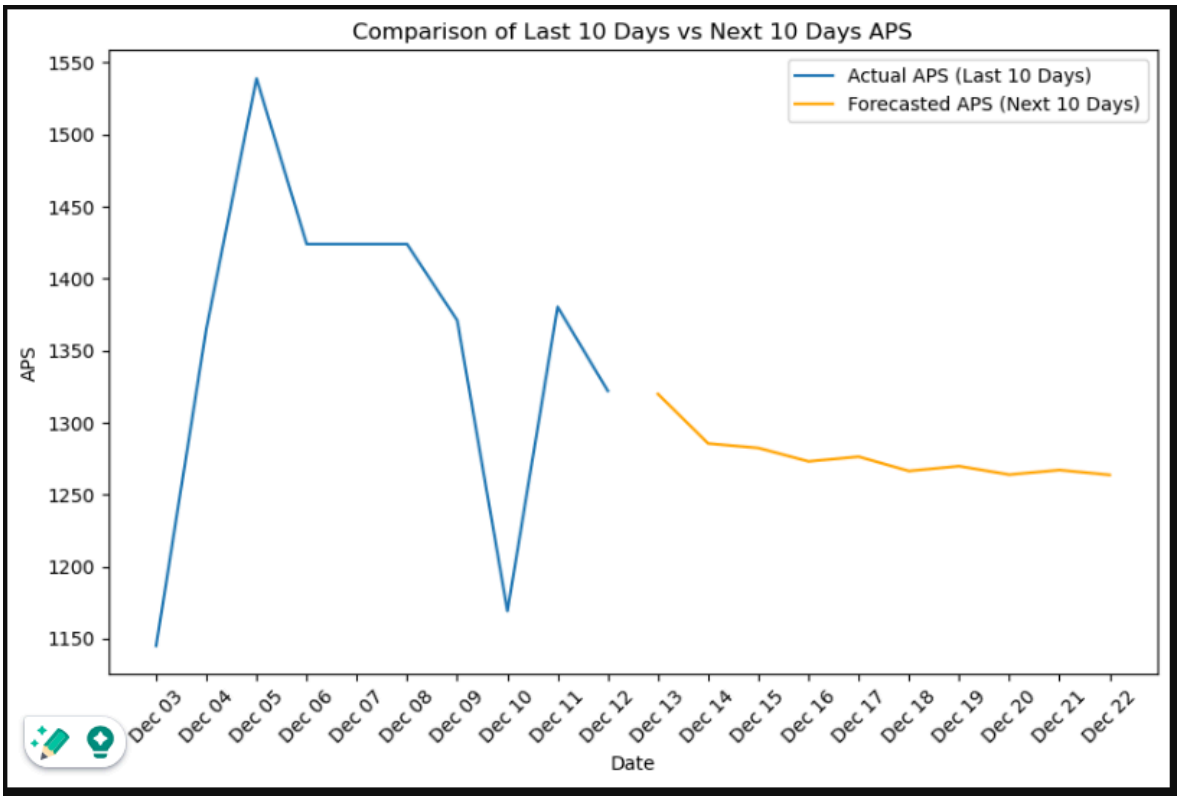
```
# Step 2: Forecast the next 10 days using the best model
forecast_steps = 10
forecast = sarima_model.predict(n_periods=forecast_steps)

# Create a DateTime index for the next 10 days
last_date = aps_df.index[-1]
forecast_dates = pd.date_range(last_date, periods=forecast_steps + 1,
freq='D')[1:]
```

```
# Create a DataFrame for the forecast results
forecast_df = pd.DataFrame({'Date': forecast_dates, 'Forecasted_APS':
forecast})
forecast_df.set_index('Date', inplace=True)

# Print the forecast
print(forecast_df)
```

```
                                Forecasted_APS
Date
2024-12-13 00:00:00+00:00      1319.826171
2024-12-14 00:00:00+00:00      1285.479650
2024-12-15 00:00:00+00:00      1282.343019
2024-12-16 00:00:00+00:00      1273.015767
2024-12-17 00:00:00+00:00      1276.297838
2024-12-18 00:00:00+00:00      1266.357831
2024-12-19 00:00:00+00:00      1269.729728
2024-12-20 00:00:00+00:00      1263.814247
2024-12-21 00:00:00+00:00      1267.012002
2024-12-22 00:00:00+00:00      1263.643334
```

```
# Step 3: Plot the actual data and forecasted values
# Get the last 10 days from the actual data
last_10_days = aps_df.tail(10)

# Combine the last 10 days of actual data with the forecasted values
combined_df = pd.concat([last_10_days, forecast_df])

plt.figure(figsize=(10, 6))
plt.plot(last_10_days.index, last_10_days['APS'], label='Actual APS
(Last 10 Days)')
plt.plot(forecast_df.index, forecast_df['Forecasted_APS'],
label='Forecasted APS (Next 10 Days)', color='orange')
plt.xlabel('Date')
plt.ylabel('APS')
plt.title('Comparison of Last 10 Days vs Next 10 Days APS')

# Format the x-axis labels: rotate and only show month and day
plt.xticks(ticks=combined_df.index, labels=[label.strftime('%b %d')
for label in combined_df.index], rotation=45)
```

```
plt.legend()
plt.show()
```




```
# Split the data into training and testing sets
train_size = int(len(aps_df) * 0.8)
train_data, test_data = aps_df[:train_size], aps_df[train_size:]

# Fit the SARIMA model to the training data
sarima_model = pm.auto_arima(train_data['APS'], seasonal=True, m=5,
stepwise=True, trace=True, error_action='ignore',
suppress_warnings=True)

# Make predictions on the test data
forecast = sarima_model.predict(len(test_data))

# Calculate MSE
mse = mean_squared_error(test_data['APS'], forecast)
print('Mean Squared Error:', mse)

# Calculate R-squared
y_true = test_data['APS']
y_pred = forecast
y_mean = np.mean(y_true)
```

```python
# Calculate the squared residuals
ssr = np.sum((y_true - y_pred) ** 2)

# Calculate the total squared deviations from the mean
sst = np.sum((y_true - y_mean) ** 2)

# Calculate R-squared
r_squared = 1 - ssr / sst
print('R-squared:', r_squared)
```

```
Performing stepwise search to minimize aic
 ARIMA(2,1,2)(1,0,1)[5] intercept   : AIC=50223.616, Time=12.71 sec
 ARIMA(0,1,0)(0,0,0)[5] intercept   : AIC=51007.637, Time=0.15 sec
 ARIMA(1,1,0)(1,0,0)[5] intercept   : AIC=50723.380, Time=1.65 sec
 ARIMA(0,1,1)(0,0,1)[5] intercept   : AIC=50617.470, Time=1.97 sec
 ARIMA(0,1,0)(0,0,0)[5]             : AIC=51005.733, Time=0.08 sec
 ARIMA(2,1,2)(0,0,1)[5] intercept   : AIC=50222.886, Time=8.10 sec
 ARIMA(2,1,2)(0,0,0)[5] intercept   : AIC=50220.903, Time=4.30 sec
 ARIMA(2,1,2)(1,0,0)[5] intercept   : AIC=50222.887, Time=5.78 sec
 ARIMA(1,1,2)(0,0,0)[5] intercept   : AIC=50230.361, Time=2.68 sec
 ARIMA(2,1,1)(0,0,0)[5] intercept   : AIC=50228.123, Time=3.37 sec
 ARIMA(3,1,2)(0,0,0)[5] intercept   : AIC=50222.891, Time=7.30 sec
 ARIMA(2,1,3)(0,0,0)[5] intercept   : AIC=50222.909, Time=4.26 sec
 ARIMA(1,1,1)(0,0,0)[5] intercept   : AIC=50245.218, Time=2.01 sec
 ARIMA(1,1,3)(0,0,0)[5] intercept   : AIC=50223.738, Time=4.49 sec
 ARIMA(3,1,1)(0,0,0)[5] intercept   : AIC=50222.982, Time=3.79 sec
 ARIMA(3,1,3)(0,0,0)[5] intercept   : AIC=50224.903, Time=6.22 sec
 ARIMA(2,1,2)(0,0,0)[5]             : AIC=50224.234, Time=1.18 sec

Best model:  ARIMA(2,1,2)(0,0,0)[5] intercept
Total fit time: 70.125 seconds
Mean Squared Error: 75829.17663419979
R-squared: -1.1526812047933954
```

```python
# Step 1: Calculate the Total Volume per Date
total_volume_per_date =
final_data_scaled.groupby('Date')['Volume'].sum()

# Step 2: Define a function to calculate the weighted aggregate for
each column (Open, Close, High, Low)
def calculate_weighted_aggregate(group, column, total_volume):
    # Calculate the weight for each row
    group['Weight'] = group['Volume'] / total_volume
    # Compute the weighted aggregate sum
    weighted_aggregate = (group['Weight'] * group[column]).sum()
    return weighted_aggregate

# Step 3: Calculate the weighted aggregates for Open, Close, High, Low
aggregated_open = final_data_scaled.groupby('Date').apply(lambda
```

```
group: calculate_weighted_aggregate(group, 'Open',
total_volume_per_date[group.name]))
aggregated_close = final_data_scaled.groupby('Date').apply(lambda
group: calculate_weighted_aggregate(group, 'Close',
total_volume_per_date[group.name]))
aggregated_high = final_data_scaled.groupby('Date').apply(lambda
group: calculate_weighted_aggregate(group, 'High',
total_volume_per_date[group.name]))
aggregated_low = final_data_scaled.groupby('Date').apply(lambda group:
calculate_weighted_aggregate(group, 'Low',
total_volume_per_date[group.name]))

# Step 4: Aggregate the Volume using sum (since we are adding up the
volumes)
aggregated_volume = final_data_scaled.groupby('Date')['Volume'].sum()

# Step 5: Extract the first value of APS, Gold Price (INR), 1 USD
(INR), and S&P 500 (INR) for each date
aggregated_aps = final_data_scaled.groupby('Date')['APS'].first()
aggregated_gold_price = final_data_scaled.groupby('Date')['Gold Price
(INR)'].first()
aggregated_usd_inr = final_data_scaled.groupby('Date')['1 USD
(INR)'].first()
aggregated_sp500_inr = final_data_scaled.groupby('Date')['S&P 500
(INR)'].first()

# Step 6: Combine the aggregated results into a new DataFrame
aggregated_data = pd.DataFrame({
    'Open': aggregated_open,
    'Close': aggregated_close,
    'High': aggregated_high,
    'Low': aggregated_low,
    'Volume': aggregated_volume,
    'APS': aggregated_aps,
    'Gold Price (INR)': aggregated_gold_price,
    '1 USD (INR)': aggregated_usd_inr,
    'S&P 500 (INR)': aggregated_sp500_inr
}).reset_index()
```

aggregated_data

	Date	Open	Close	High	Low	Volume	APS	Gold Price (INR)	1 USD (INR)	S&P 500 (INR)
0	2008-08-28 00:00:00+00:00	124.803408	122.117572	125.384411	121.224185	40017944.0	90.334791	36268.787334	43.650002	56774.686329
1	2008-08-29 00:00:00+00:00	160.815515	164.376914	164.829385	160.613270	16657102.0	118.779990	36178.211967	43.625000	55963.456833
2	2008-08-30 00:00:00+00:00	189.250000	186.887497	189.250000	184.250000	1402980.0	118.779990	36178.211967	43.625000	55963.456833
3	2008-08-31 00:00:00+00:00	189.250000	186.887497	189.250000	184.250000	1402980.0	118.779990	36178.211967	43.625000	55963.456833
4	2008-09-01 00:00:00+00:00	189.250000	186.887497	189.250000	184.250000	1402980.0	118.779990	36178.211967	43.625000	55963.456833
...	...	...	...	...	...	...	...	...	...	...
5946	2024-12-08 00:00:00+00:00	1797.000000	1782.800049	1797.000000	1775.599976	5904200.0	1423.916884	223446.682274	84.683800	515747.206719
5947	2024-12-09 00:00:00+00:00	1364.560960	1370.943446	1378.054046	1354.294164	18075676.0	1370.943446	225626.416108	84.666000	512470.608585
5948	2024-12-10 00:00:00+00:00	1163.758752	1169.074821	1179.941761	1158.747107	29612049.0	1169.074821	228884.078476	84.847298	512045.818430
5949	2024-12-11 00:00:00+00:00	1374.763836	1380.336477	1384.578355	1368.547520	17942045.0	1380.336477	232028.546846	84.874001	516389.540429
5950	2024-12-12 00:00:00+00:00	1313.384254	1321.980742	1332.440344	1310.607940	26657308.0	1321.980742	229522.542132	84.822998	515431.172215

```
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split


# Define the predict_future function
def predict_future(model, last_sequence, scaler, features, time_steps,
days_to_predict):
    predicted_aps = []
    input_seq = last_sequence.reshape(1, time_steps, len(features))

    for _ in range(days_to_predict):
        # Predict the next APS value
        next_aps_scaled = model.predict(input_seq)

        # Inverse transform to get the actual APS value
        dummy_array = np.zeros((1, len(features)))
        dummy_array[0, -1] = next_aps_scaled[0, 0]
        next_aps_original = scaler.inverse_transform(dummy_array)[0,
-1]

        predicted_aps.append(next_aps_original)

        # Prepare the input for the next prediction
        input_seq = np.roll(input_seq, -1, axis=1)
        input_seq[0, -1, :-1] = last_sequence[-1, :-1]  # Keep other
features the same
        input_seq[0, -1, -1] = next_aps_scaled[0, 0]  # Use scaled
prediction

    return np.array(predicted_aps)
```

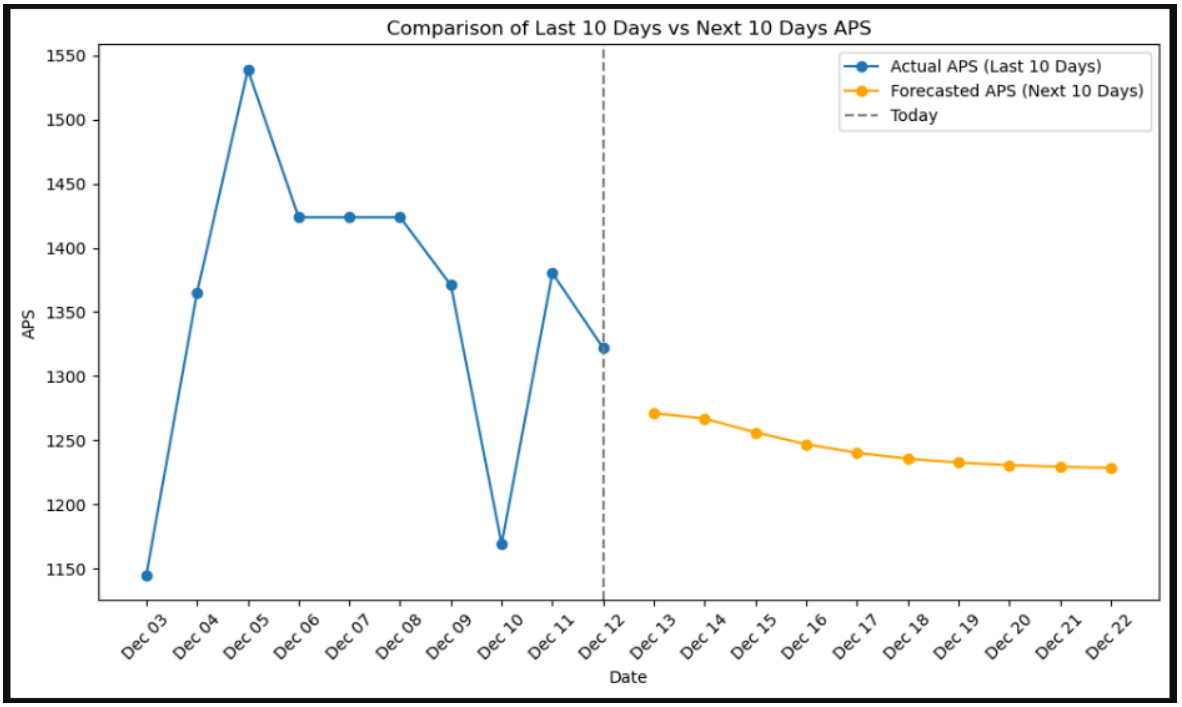
```
# Create a separate scaler for the target variable
target_scaler = MinMaxScaler()
aggregated_data['APS_scaled'] =
target_scaler.fit_transform(aggregated_data[['APS']])

# Modify the scaling process
features = ['Open', 'Close', 'High', 'Low', 'Volume', 'Gold Price
(INR)', '1 USD (INR)', 'S&P 500 (INR)']
feature_scaler = MinMaxScaler()
scaled_features =
feature_scaler.fit_transform(aggregated_data[features])

# Create a combined scaled dataset
scaled_data = np.column_stack([
    scaled_features,
    aggregated_data['APS_scaled'].values
])

# Create sequences (rest of your existing sequence creation code
remains the same)
def create_sequences(data, time_steps=60):
    X = []
    y = []
    for i in range(time_steps, len(data)):
        X.append(data[i-time_steps:i, :-1])  # Exclude last column
(target)
        y.append(data[i, -1])  # Target value
    return np.array(X), np.array(y)

# Create sequences for training
time_steps = 60
X, y = create_sequences(scaled_data, time_steps)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, shuffle=False)

# Define the GRU Model
model = Sequential([
    GRU(units=64, return_sequences=False,
input_shape=(X_train.shape[1], X_train.shape[2])),
    Dense(units=1)
])

# Compile the model
```

```
model.compile(optimizer=Adam(learning_rate=0.001),
loss='mean_squared_error')

# Train with early stopping
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_data=(X_test, y_test),
    callbacks=[early_stopping]
)

# Prepare the last sequence for prediction
last_sequence = scaled_data[-time_steps:, :-1]

# Make predictions
predicted_aps_scaled = predict_future(model, last_sequence,
target_scaler, features, time_steps, 10)

# Inverse transform the predictions
predicted_aps_original = predicted_aps_scaled

print("Predicted APS values:", predicted_aps_original)
```

```
Epoch 1/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 11s 47ms/step - loss: 0.0054 - val_loss: 0.0065
Epoch 2/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 7s 44ms/step - loss: 0.0011 - val_loss: 0.0071
Epoch 3/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 7s 45ms/step - loss: 0.0011 - val_loss: 0.0065
Epoch 4/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 6s 44ms/step - loss: 0.0011 - val_loss: 0.0066
Epoch 5/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 7s 44ms/step - loss: 0.0010 - val_loss: 0.0066
Epoch 6/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 7s 45ms/step - loss: 0.0011 - val_loss: 0.0070
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 387ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 67ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
Predicted APS values: [1271.0360255  1266.81078681 1256.12823337 1246.86703799 1240.15385948
 1235.58231965 1232.54950265 1230.55859292 1229.25653067 1228.40483155]
```

```python
# Ensure the 'Date' column is properly set as datetime
if 'Date' in aggregated_data.columns:
    aggregated_data['Date'] = pd.to_datetime(aggregated_data['Date'])
    aggregated_data.set_index('Date', inplace=True)

# Get the last 10 dates and actual APS values
last_10_actual = aggregated_data['APS'][-10:].values
last_10_dates = aggregated_data.index[-10:]

# Generate the next 10 dates for the forecast
last_date = last_10_dates[-1]
forecast_Ddates = pd.date_range(start=last_date +
pd.Timedelta(days=1), periods=10)

# Combine the data
all_dates = list(last_10_dates) + list(forecast_dates)
all_values = list(last_10_actual) + list(predicted_aps_original)

# Plot the data
plt.figure(figsize=(10, 6))
plt.plot(last_10_dates, last_10_actual, label='Actual APS (Last 10
Days)', marker='o')
plt.plot(forecast_dates, predicted_aps_original, label='Forecasted APS
(Next 10 Days)', marker='o', color='orange')
plt.axvline(last_date, color='gray', linestyle='--', label='Today')
plt.xlabel('Date')
plt.ylabel('APS')
```

```
plt.title('Comparison of Last 10 Days vs Next 10 Days APS')

# Format the x-axis: Show only month and day, rotated
formatted_dates = [date.strftime('%b %d') for date in all_dates]
plt.xticks(ticks=all_dates, labels=formatted_dates, rotation=45)

plt.legend()
plt.tight_layout()
plt.show()
```




```
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
# Perform train-test split on the entire dataset
train_size = int(len(aggregated_data) * 0.8)
train_data = aggregated_data[:train_size]
test_data = aggregated_data[train_size:]

# Create separate scalers for training data
target_scaler = MinMaxScaler()
feature_scaler = MinMaxScaler()

# Scale training features
train_features = ['Open', 'Close', 'High', 'Low', 'Volume', 'Gold
Price (INR)', '1 USD (INR)', 'S&P 500 (INR)']
train_scaled_features =
feature_scaler.fit_transform(train_data[train_features])
train_scaled_target = target_scaler.fit_transform(train_data[['APS']])
```

```
# Combine scaled training data
train_scaled_data = np.column_stack([
    train_scaled_features,
    train_scaled_target
])

# Create training sequences
time_steps = 60
X_train, y_train = create_sequences(train_scaled_data, time_steps)

# Scale test features using the same scalers
test_scaled_features =
feature_scaler.transform(test_data[train_features])
test_scaled_target = target_scaler.transform(test_data[['APS']])

# Combine scaled test data
test_scaled_data = np.column_stack([
    test_scaled_features,
    test_scaled_target
])

# Create test sequences
X_test, y_test = create_sequences(test_scaled_data, time_steps)

# Define the GRU Model
model = Sequential([
    GRU(units=64, return_sequences=False,
input_shape=(X_train.shape[1], X_train.shape[2])),
    Dense(units=1)
])

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.001),
loss='mean_squared_error')

# Train with early stopping
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
```

```
    validation_split=0.2,  # Use validation split instead of separate
test data
    callbacks=[early_stopping]
)

# Evaluation function
def evaluate_model(model, X_test, y_test, target_scaler):
    # Predict
    y_pred_scaled = model.predict(X_test)
    y_pred = target_scaler.inverse_transform(y_pred_scaled)
    y_test_original =
target_scaler.inverse_transform(y_test.reshape(-1, 1))

    # Calculate metrics
    mse = mean_squared_error(y_test_original, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test_original, y_pred)
    mape = np.mean(np.abs((y_test_original - y_pred) /
y_test_original)) * 100
    r2 = r2_score(y_test_original, y_pred)

    print("Model Evaluation Metrics:")
    print(f"MSE: {mse}")
    print(f"RMSE: {rmse}")
    print(f"MAE: {mae}")
    print(f"MAPE: {mape}%")
    print(f"R-squared: {r2}")

    return {
        'MSE': mse,
        'RMSE': rmse,
        'MAE': mae,
        'MAPE': mape,
        'R-squared': r2
    }

# Evaluate the model
metrics = evaluate_model(model, X_test, y_test, target_scaler)
```

```
Epoch 1/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 16s 79ms/step - loss: 0.0026 - val_loss: 0.0062
Epoch 2/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 69ms/step - loss: 0.0010 - val_loss: 0.0054
Epoch 3/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 10s 68ms/step - loss: 0.0011 - val_loss: 0.0058
Epoch 4/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 70ms/step - loss: 0.0011 - val_loss: 0.0054
Epoch 5/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 69ms/step - loss: 0.0010 - val_loss: 0.0054
Epoch 6/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 9s 74ms/step - loss: 9.9759e-04 - val_loss: 0.0055
Epoch 7/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 7s 63ms/step - loss: 9.7045e-04 - val_loss: 0.0053
Epoch 8/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 67ms/step - loss: 9.8411e-04 - val_loss: 0.0054
Epoch 9/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 67ms/step - loss: 9.2687e-04 - val_loss: 0.0053
Epoch 10/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 70ms/step - loss: 0.0012 - val_loss: 0.0051
Epoch 11/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 10s 64ms/step - loss: 0.0011 - val_loss: 0.0050
Epoch 12/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 10s 65ms/step - loss: 8.9571e-04 - val_loss: 0.0050
Epoch 13/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 66ms/step - loss: 0.0010 - val_loss: 0.0052
Epoch 14/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 67ms/step - loss: 9.5424e-04 - val_loss: 0.0050
Epoch 15/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 64ms/step - loss: 0.0011 - val_loss: 0.0048
Epoch 16/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 66ms/step - loss: 8.8183e-04 - val_loss: 0.0049
Epoch 17/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 11s 69ms/step - loss: 9.2679e-04 - val_loss: 0.0049
Epoch 18/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 9s 73ms/step - loss: 8.6125e-04 - val_loss: 0.0047
Epoch 19/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 71ms/step - loss: 9.1984e-04 - val_loss: 0.0048
Epoch 20/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 9s 72ms/step - loss: 0.0010 - val_loss: 0.0048
Epoch 21/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - loss: 9.2647e-04 - val_loss: 0.0051
Epoch 22/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 8s 72ms/step - loss: 8.5898e-04 - val_loss: 0.0051
Epoch 23/50
118/118 ━━━━━━━━━━━━━━━━━━━━ 9s 73ms/step - loss: 7.8893e-04 - val_loss: 0.0048
36/36 ━━━━━━━━━━━━━━━━━━━━ 2s 42ms/step
Model Evaluation Metrics:
MSE: 18856.14992634236
RMSE: 137.31769706174933
MAE: 98.70005621258956
MAPE: 10.59885366595499%
R-squared: 0.4672231671527898
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.callbacks import EarlyStopping
# Define the predict_future function for LSTM
def predict_future_lstm(model, last_sequence, scaler, features,
time_steps, days_to_predict):
    predicted_aps = []
    input_seq = last_sequence.reshape(1, time_steps, len(features))

    for _ in range(days_to_predict):
        # Predict the next APS value
        next_aps_scaled = model.predict(input_seq)

        # Inverse transform to get the actual APS value
        dummy_array = np.zeros((1, len(features)))
        dummy_array[0, -1] = next_aps_scaled[0, 0]
        next_aps_original = scaler.inverse_transform(dummy_array)[0,
-1]

        predicted_aps.append(next_aps_original)

        # Prepare the input for the next prediction
        input_seq = np.roll(input_seq, -1, axis=1)
        input_seq[0, -1, :-1] = last_sequence[-1, :-1]  # Keep other
features the same
        input_seq[0, -1, -1] = next_aps_scaled[0, 0]  # Use scaled
prediction

    return np.array(predicted_aps)

# Create a separate scaler for the target variable
target_scaler = MinMaxScaler()
aggregated_data['APS_scaled'] =
target_scaler.fit_transform(aggregated_data[['APS']])

# Modify the scaling process
features = ['Open', 'Close', 'High', 'Low', 'Volume', 'Gold Price
(INR)', '1 USD (INR)', 'S&P 500 (INR)']
feature_scaler = MinMaxScaler()
scaled_features =
feature_scaler.fit_transform(aggregated_data[features])

# Create a combined scaled dataset
scaled_data = np.column_stack([
    scaled_features,
    aggregated_data['APS_scaled'].values
```

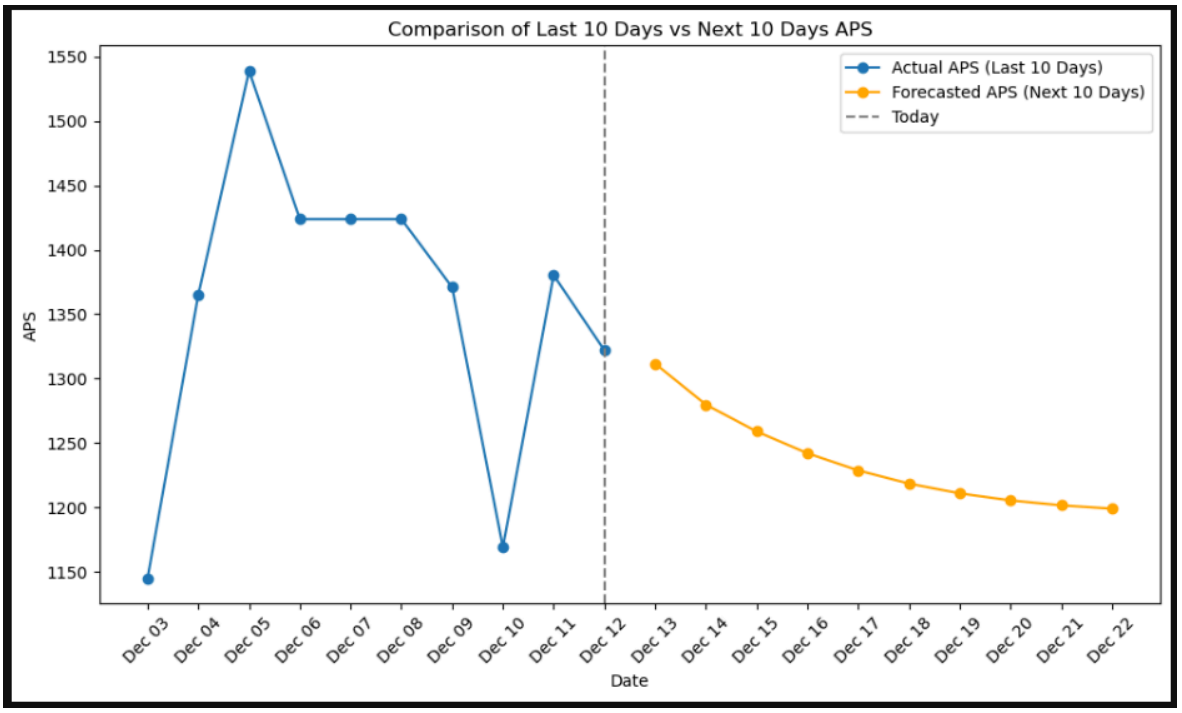
```
])

# Create sequences
def create_sequences(data, time_steps=60):
    X = []
    y = []
    for i in range(time_steps, len(data)):
        X.append(data[i-time_steps:i, :-1])  # Exclude last column
(target)
        y.append(data[i, -1])  # Target value
    return np.array(X), np.array(y)

# Create sequences for training
time_steps = 60
X, y = create_sequences(scaled_data, time_steps)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, shuffle=False)

# Define the LSTM Model
model = Sequential([
    LSTM(units=64, return_sequences=False,
input_shape=(X_train.shape[1], X_train.shape[2])),
    Dense(units=1)
])

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Train with early stopping
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_data=(X_test, y_test),
    callbacks=[early_stopping]
)
```

```
Epoch 1/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 16s 74ms/step - loss: 0.0033 - val_loss: 0.0076
Epoch 2/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 18s 58ms/step - loss: 0.0011 - val_loss: 0.0077
Epoch 3/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 11s 61ms/step - loss: 0.0011 - val_loss: 0.0090
Epoch 4/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 10s 64ms/step - loss: 0.0010 - val_loss: 0.0069
Epoch 5/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 11s 71ms/step - loss: 0.0010 - val_loss: 0.0099
Epoch 6/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - loss: 0.0010 - val_loss: 0.0067
Epoch 7/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - loss: 0.0011 - val_loss: 0.0065
Epoch 8/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 10s 69ms/step - loss: 9.8915e-04 - val_loss: 0.0069
Epoch 9/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 10s 64ms/step - loss: 9.1604e-04 - val_loss: 0.0064
Epoch 10/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 10s 69ms/step - loss: 0.0011 - val_loss: 0.0069
Epoch 11/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 10s 69ms/step - loss: 0.0010 - val_loss: 0.0077
Epoch 12/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - loss: 0.0011 - val_loss: 0.0065
Epoch 13/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 18s 53ms/step - loss: 9.8804e-04 - val_loss: 0.0095
Epoch 14/50
148/148 ━━━━━━━━━━━━━━━━━━━━ 12s 63ms/step - loss: 0.0010 - val_loss: 0.0084
```

```
# Prepare the last sequence for prediction
last_sequence = scaled_data[-time_steps:, :-1]

# Make predictions
predicted_aps_scaled = predict_future_lstm(model, last_sequence,
target_scaler, features, time_steps, 10)

# Inverse transform the predictions
predicted_aps_original = predicted_aps_scaled

print("Predicted APS values:", predicted_aps_original)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 502ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 86ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 104ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 89ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 98ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 99ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 81ms/step
Predicted APS values: [1311.57922096 1279.73292595 1258.8954241  1241.99200191 1228.63484073
 1218.41187654 1210.81535118 1205.33235704 1201.49858455 1198.91634524]
```

```
# Ensure the 'Date' column is properly set as datetime
if 'Date' in aggregated_data.columns:
    aggregated_data['Date'] = pd.to_datetime(aggregated_data['Date'])
    aggregated_data.set_index('Date', inplace=True)

# Get the last 10 dates and actual APS values
last_10_actual = aggregated_data['APS'][-10:].values
last_10_dates = aggregated_data.index[-10:]

# Generate the next 10 dates for the forecast
last_date = last_10_dates[-1]
forecast_dates = pd.date_range(start=last_date + pd.Timedelta(days=1),
periods=10)

# Combine the data
all_dates = list(last_10_dates) + list(forecast_dates)
all_values = list(last_10_actual) + list(predicted_aps_original)

# Plot the data
plt.figure(figsize=(10, 6))
plt.plot(last_10_dates, last_10_actual, label='Actual APS (Last 10 Days)',
marker='o')
plt.plot(forecast_dates, predicted_aps_original, label='Forecasted APS (Next
10 Days)', marker='o', color='orange')
plt.axvline(last_date, color='gray', linestyle='--', label='Today')
plt.xlabel('Date')
plt.ylabel('APS')
plt.title('Comparison of Last 10 Days vs Next 10 Days APS')

# Format the x-axis: Show only month and day, rotated
formatted_dates = [date.strftime('%b %d') for date in all_dates]
plt.xticks(ticks=all_dates, labels=formatted_dates, rotation=45)

plt.legend()
plt.tight_layout()
plt.show()
```




```
y_pred = model.predict(X_test)
y_pred_original = target_scaler.inverse_transform(
    np.column_stack([np.zeros((y_pred.shape[0], len(features))), y_pred])
)[:, -1]
y_test_original = target_scaler.inverse_transform(
    np.column_stack([np.zeros((y_test.shape[0], len(features))),
y_test.reshape(-1, 1)])
)[:, -1]

mse = mean_squared_error(y_test_original, y_pred_original)
mae = mean_absolute_error(y_test_original, y_pred_original)
mape = mean_absolute_percentage_error(y_test_original, y_pred_original)
r2 = r2_score(y_test_original, y_pred_original)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Mean Absolute Percentage Error (MAPE):", mape * 100)
print("R^2 Score:", r2)
```

10.2 PLAGIARISM REPORT

11. REFERENCES

[1] ***Mohan, B., Yadav, A., Toleti, J., Reddy, D. S. S., Ahmed, R. K., & Koushitha, G. (2024).*** *Sentiment-driven predictive models for Nifty 50 stock market fluctuations: A time series analysis perspective. IEEE International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS), 70, 1–8.* https://doi.org/10.1109/iciteics61368.2024.10625537

[2] **Panchal, S. A., Ferdouse, L., & Sultana, A. (2024).** Comparative analysis of ARIMA and LSTM models for stock price prediction. *2024 IEEE/ACIS 27th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, 240–244. https://doi.org/10.1109/snpd61259.2024.10673919

[3] **Rajhans, A., Das, T., Kumar, A., & Sinha, B. B. (2024).** Exploring the impact of financial news sentiment on stock price forecasting: A comparative deep learning approach. *2024 1st International Conference on Communications and Computer Science (InCCCS).* https://doi.org/10.1109/incccs60947.2024.10593315

[4] **Taslim, D. G., Tjahyadi, H., & Murwantara, I. M. (2024).** A study of LSTM optimisation for forecasting volatile time series data. *2024 4th International Conference on Computer Communication and Artificial Intelligence (CCAI).* https://doi.org/10.1109/ccai61966.2024.10603183

[5] **Amruth, S. J., Nigelesh, T., Shruthik, V. S., Reddy, V. S., & Venugopalan, M. (2024).** Time-series-based stock market analysis using machine learning. *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), 1–7.* https://doi.org/10.1109/icccnt61001.2024.10725232

12. WORKLOG

Day	Date	Task Done
Day 1	11/11/2024	Domain Selection, Base Paper Selection, Finalising the TItle and Problem Statement
Day 2	12/11/2024	Final Abstract (Objective, Dataset methodology, Tools, Expected Outcome) and Review of Related Literature
Day 3	13/11/2024	Approach to the Problem write up (Aim, Research Objective, Research questions, Algorithm, Tools, Dataset, proposed architecture, expected outcome, References). Performing EDA
Day 4	14/11/2024	Approach to the Problem write up (Aim, Research Objective, Research questions, Algorithm, Tools, Dataset, proposed architecture, expected outcome, References). Performing EDA
Day 5	18/11/2024	Implementation
Day 6	19/11/2024	Implementation
Day 7	20/11/2024	Implementation
Day 8	21/11/2024	Implementation
Day 9	22/11/2024	Second Review
Day 10	25/11/2024	Results and Discussion
Day 11	29/11/2023	Conclusion and future scope
Day 12	3/12/2023	Draft of report submission
Day 13	6/12/2024	PPT for Final Review Submission
Day 14	10/12/2024	PPT for Final Review Submission and Project Report Submission on approval by Project Mentor
Day 15	13/12/2024	Final Review